

EUROPEAN UNIVERSITY OF LEFKE

FACULTY OF ENGINEERING

Graduation Project 2

Nobel Source

Bruno SHEMA

22243402

Nobel source is a trade coordination platform designed to solve a symmetry in agricultural and other raw material supplying chain. The platform enables suppliers and buyers to come together and agree on business term; price, volume, dates of delivery, transportation, method of payment,... many relevant factors. Yes there are services doing these, but are scattered around, and they are price what Nobel source offers, is an intuitive, way of doing what we used to call tedious work. What created the gap we have now. What is making middleman layer increase uncontrollably just to benefit the lack of information.

Main idea is : if both sides see the same information, exploitation become harder. The platform does not handle payments, does not provide legal enforcement, and does not claim to eliminate market crashes. It solves one core problem only; the gap between what one side I knows and what the other does not know. The goal are not to reduce price volatility, through pre-agreed terms, provide proof of delivery through timestamped confirmations, create visible trust scores from completed deals and give farmers predictable income before they invest in production.

This will allow people to do business not as gamble but as a well structured approach, with a clear goal and easy to read the signs of the market as everything is visible at the right time. for the right view. We need to trade with nobility.

Supervisor

Dr. Vesile Evrim

12/05/2026

Table Of Contents

Bruno SHEMA	1
22243402	1
1.Introduction	1
1.1 Problem definition	1
1.1.1 The motivation to the idea	1
1.1.2 The Three core problems	2
1.1.3 Why no one has solved this yet ?	3
1.1.4 What Gaps do we fill ? Who do we compete? How do we win?	4
1.2 Strategic framework	5
1.2.1 Goals	5
1.2.2 Objectives	6
1.2.3 Out of scope (idea wise)	7
1.2.4 Success metrics	8
2. Literature Survey	9
Comparision 1: Traditional Marketplace platforms	9
Comparision 2: Logistic Management Systems (Freightos ShipBob)	10
Comparision 3: Messaging -first trade platform(whatsapp business, wechat)	11
Comparision 4: Enterprise Enterprise Resource Planning Modules (SAP trade Management , Oracle SCM)	11
Comparison 5: Active agritech startups	12
3. Background Information	12
3.1 Technological Adoption Strategy & Learning Curve	13
3.2 Required & Used Software	13
3.3 Graphics & runtime tools	17
3.4 Development tools	17
3.5 Hardware Environment profiles	17
4. Design Documents	18
4.1 Data flow diagram - Level 0 (context diagram)	18
4.2 Nobel Source -Use case Diagram	21
4.3 Nobel Source -ER Diagram (10 tables)	22
4.3.1 ER Diagram explained	22
4.3.2 Core Design Idea	23

4.3.2 Tables	23
5. Methodology	30
5. 1 Development Context and engineering approach	30
5. 2 Database Environment setup and initialization	31
5. 3 Schema Design Evolution	31
5. 3.1 Transition to JSONB- based flexibility layer	32
5. 4 Schema Rest Strategy	33
5. 5 Entity-centered Architecture	34
5. 6 Backend Architecture	35
5. 6.1 database connection	35
5. 6.2 design schema	37
5. 6.2 Seed your database	41
5. 6.2 Seed your database	42
5.6.3 create your services	43
5.6.4 create your controllers	48
5.6.5 create your route	51
5.6.5 Serve apis, & api documentation	53
5. 7 Frontend Architecture	54
So we have in the backend to submit a formdata, when it involve media, or a normal json ,	57
6. Website Overview	60
7. Conclusion	69
7.1 Benefits	69
a. Benefits to users :	69
b. Benefits to me :	70
7.2 Ethics	70
7.3 Sustainable Development Goals (SDGs)	Error! Bookmark not defined.
7.4 Commercial potential	Error! Bookmark not defined.
7.5 Future Works	Error! Bookmark not defined.
8. References	75

1.Introduction

1.1 Problem definition

1.1.1 The motivation to the idea

Needs vs Luxury

Why can I order a \$5 phone case from China, track it across an ocean receive it and return it if it's wrong size- but I cannot do the same of \$1,000 of vegetables from a farmer 50 kilometers away?

Insight: it seem for smaller items we came to accept that it is safe to order them, from a far, even to relaxed to order them from just down stairs; but this was not always the case; they had time to transition as well. They were skeptical as well; like an online payment, the hacker can intercept the transaction, we don't physically see the item , they might bring ill-sized items.... more of these worries

They were not wrong, but later answers we also found, block-chain technology, the return policy; all these happened because they gave a chance to the technology and every one now testify the use of online, every business is eager to operate, online.

Another fact why people are not open to food supply. Is large quantity, they fear to risk it. But as the solutions we found, the same situation ; these money are not risked they will be secured.

In short, high-value high risk essential goods operate with less infrastructure than low-value , low-risk luxury goods. The result is repeated cycles of price crashes , farmers exploitation and broken trust, as we are leaning on less value , not with intention to fix the system , but to coop with it, we should stress test the working system so to fix the every bug, it works for small value, why can't it work for high value. Or it never really worked!!

Why suppliers always keep complaining, buyer nothing heard from them?

“Suppliers are often expected to meet stringed delivery schedules and quality standards, even when external factors such as supply chain disruption or economic fluctuations interfere with their progress”[31].this is very much true, the suppliers are the one we expect a lot from , yet they literally have not up hand in the market decisions. The same reason maybe stated here, the miscommunication between the two parties.

We can also say that this arise from:

1. Contract rigidity: standard contracts we have now days , typically favor one party, often leaving little room for the negotiation or flexibility when unforeseen challenges arise, which happened often in this domain of large quantity supply.

2. Greedy mentality most have for the market: strategic partnerships are most frequently over picked by short time transaction, business are familiar with just today transaction , not for long time partnership.This show that the culture of strategic planning is not favored by the current system as it has no room, yet the education system encourage so, literature proves the benefits of it; and we are not practicing it at all.Only if business can take supplier as strategic partners rather than mere vendors, the growth in this industry will yield a better outcome.

3. Flexible and adaptive contract; this seem like even asking for much, if they are no communication even; it is only a dream to have a flexible contract. Yet one well known factor is that the world is spontaneous, simply we can not predict the future to the perfection. If the rules that guide us are not flexible, yet the path we walk in changes frequently, obviously we are being misled.

1.1.2 The Three core problems

1. Price Volatility

Where I work Turkish Republic of Northern Cyprus, we peek Cucumbers, the prices reduce dramatically from the day we start harvesting till the end of the season which is normal; it mark the pattern of the market. What shock me in summer of 2025, they cucumber price came from 100TL to 3 TL in just 3 weeks. This is 97% price loss. We can not even label it as a market shift as this is a huge jump not what market can produce in 3 weeks.

The bottom line cause is market separation. And unstructured plan, and unable to read signs of the market due to lack of information on time. The suppliers lack market information as they are not the one to interaction with it. The middleman who are in between exploit and take advantage of the missing communication.

In UK they had a similar case; as recorded by UK's parliament, on potatoes market, noting that price swings of over 90% have been normalized despite being economically destructive.

2. Middlemen layers Capture Value without adding it

In the Philippines, five to six layers of intermediaries stand between the milkfish farmer and the consumer. Each layer takes a margin. The retail prices nearly double the farmgate price but the farmer sees more of that increases[3].

Now is really 6 layer needed? what are the role of middlemen? Some of them are really needed and they play a crucial role but when it come to 6 layers, it is no longer people helping to connect suppliers and buyers. It is a loop hole, that people are taking advantage of. These layers they cut of the payment from the buyer intended to reach the supplier. Hence the supplier always being wronged.

In India, the international Fund for Agricultural Development(IFAD) found that the intermediaries capture approximately 40% of the value that should flow at farmers[4]. This is not because intermediaries provide essential service. In many cases, they simply exploit as I said above, the lack of direct communication channels between producer and buyer. It is killing the market place. What is bothering is that we still like we need to adapt instead of finding the solution that works.

3. No Shared Record of Truth

When a shipment arrives with quantity discrepancies, the buyer says "you sent less!" and the supplier claims state "I sent exactly what we agreed!". Now who is telling the truth; without a shared record, disputes are destined to arise or some party will consistently have to abide with the other one's reasonable and non-reasonable claims. It should not be like this; when a buyer pays he should be agreeing with his conscience to the equivalency of the money paid and to the service received, and the supplier should trust that the money will actually reach their side. If shipped buyer knows it will arrive in the expected state, if supplier sent good, they know they will be paid for it.

Human Sciences Research Council of South Africa reported that 85% of smallholder farmers lack access to market information systems. Meaning they sell based on whatever price the first buyer came with, no

eyes on other alternatives, which might please them[5]. reason why later if they found out they claim they are being cheated on.

1.1.3 Why no one has solved this yet ?

This would take many forms as we tend to create solution given to the environment we grew in, and how exposed to the very same problem; and unfortunately the people in this field of work , are mostly with less education background or even none. As some people in growing country they experience child labour, in agricultural and mining fields, and the education is pricey, these very same people if they thrive as much as they can mostly they barely do secondary school and keep doing the family work. Why in agriculture most did not get enough chance to pursue high education.

Why these? To yield an answer to the current problem, you need to understand the issue. And actually be able to think outside of it. And thinking outside the problem that is what the farmers lack. And the current not many scholars have experienced the problem; so they only appreciate what comes from it .

Me I would say I was fortunate to work , in the garden ever since I reached the island, I learnt a lot; at first I had to accept everything as they are, but now I see things need a fix, and here I am trying to use my engineering knowledge to make my contribution, into the field that feeds me the whole time on the island; and feeding whoever alive.

But going a bit technical these maybe the key reasons, why even those who try, fails

1. Fresh Food Margins are Thin

Several agritech startups have attempted similar solutions and failed. Otipy, as fresh food commerce platform, collapsed because the margin on fresh produce(around 5 to 10%) can not support complex logistics and customer acquisition costs[6]. Nobel Source avoids this trap by not handling logistics, at least for now! The platform only records what happens. Farmers and buyers arrange transport themselves at their own emergency. The platform adds recording-keeping overhead, not physical movement overhead.

This will make sure that familiar with the proper care of their product they can maximize the quality conservation, rather than they money making company with less, or limited knowledge of the items on the move; can cater for them and reach the destination in good, state as these are with very thin Margins. And any human caused or natural caused mistake or factor can destroy a lot, meaning millions at a waist. We Nobel Source we don't claim to understand the gravity and the environment, so we help the system user to seamlessly handle things in their knowledge of the field.

2. Contract Enforcement is Expensive

With the high rate of not being able to attend good education, anything that is expensive will mostly luckily not be welcomed; and the fact that they possess limited education , they can not easily see the bigger picture hence don't we should not expect them to invest blindly into things that will yield result and a doubtful rate in an unknown later date. Simply they need clear and irresistible offer, which will add on to what they have not deducted .

In China; the courts have documented numerous cases where market price moved, one party breached and the other could not afford legal action[7]. Simply the very same rule that was meant to help them, is the one destroying them without any saying to it what so ever. Nobel source does not pretend to solve enforcement. It provides evidence that makes enforcement easier, but the platform explicitly states: "We

are not a legal system. We are a record-keeping system” . As our aiming problem is to close the gap by solving information sharing, we don't really go beyond that; as a lot goes on, there and if we force ourselves right away , the foundation will not be strong, hence building a system meant to fall like all the others.

3. Existing platform prioritize capture Over service

We do get that a business without benefiting the owner, is either it is doing so illegally ,in the dark or it is a wrong business idea meant to fail, with a lot of wasted money, and it would not be giving the value to the monetary system, that it deserve. This is not about being against charity but even charity should have an aim; in case the aim is not clear, it leaves unsettling feeling.

But we are rushing to money quite fast, in this delicate field. Major agritech platforms have been built with venture capital expectations of rapid scaling and monopoly returns. This creates pressure to capture users, extract data and monetize attention rather than solve the core coordination problem[8]. Nobel Source is designed as a shared tool, not a captured marketplace. The name itself “source” indicates that the producer is the center not the platform. Once we serve them right, they will onward reward from what they earned, in categories to fit everyone's potentials

1.1.4 What Gaps do we fill ? Who do we compete? How do we win?

The Nobel Source idea is mainly targeting 4 production area at the same time:

- **Messaging:** Obviously we can't solve communication if they can not every send messages to each other.
- **Public exposure(posting):** we can only communicate with the one we know, so we need to allow them to find each other, and decide if they are interested before even going it to messages.
- **Logistic:** as we are talking about items in tones, the movement of these goods will not be ignorable factor, visibility in the logistic process is the key. And by stating visibility. At least knowing what , if from where , in which method of transportation; who to hold accountable, for they goods.
- **Money Transaction:** goods of value, never separate with our monetary system, as that is how officially humans recognize formal value exchange. So we Nobel source we have a claim in this field as well.
- **Contract and legal documents:** business is mostly known from its serious, maneuvers ; no trust is built on air; legal document protects you and under the protect ; you get the opportunity to trust your partner in the legal frame.

For Potential Competitors and what they miss , our key in door passage.

Alibaba:

strong in user base and have escrow; which check the public exposure and Money transaction. The weakens is in lack of the structured shipment, means you get to be in contact with the individual , you can pay, but hopefully if the item reaches. Nobel source cover this gap by allowing both parties mark status of the goods in logistic track, giving the impression of what is going on. A key factor in trust , a pillar to effective communication.

The Entireprise resource planing (like Freightos):

Logistic optimization, with cool features, mainly design to assist in every step, from collection location to the destination. But the problem here we are transporting what we are not sure if they are paid for or they will be paid for; means for them they care , it reach the destination, I get paid and am off; And I is expensive even. Nobel Source, is built with the contract feature, make sure that whoever is consuming our service won't be tricked by any party, as well we are not leaving logistic out, we just do not stress it too much, only make sure the visibility is there, hence being cheap.

Whatsapp:

we all agree that this is the beast of messaging field for quite a long time, making a huge check on messaging, and this comes with no training unlike all other mentioned tools. But messaging is nearly only the strong factor it has, that can benefit us. It can simulate other like public exposure in status posting. And ordering. But that is not that common yet; and it is a relaxed tool. We can not expect user to be in serious mode while using it. A down factor it has structure in its data, no chart, not easy data digestion, simply not meant for business level collaboration.

The Contract farming agreements

it does well the legal documentation which is a check for them, but that is it. It has no digital tracking, rather than communicating, we are only legal securing ourselves. And in the end this feature as it is ; it is expensive by nature, when it comes to enforcing it. Nobel Source allow the contract binding but flexible enough in between the two parties, and less expensive as they engage in what they can afford. On their own will.

As you can see Nobel Source try to sit in the gap between these solutions; but ensures to be lighter than ERP, more structured than Whatsapp, cheaper than legal contracts and more complete than listings or other form of public exposure.

1.2 Strategic framework

1.2.1 Goals

The primary goal is to solve the information asymmetry; making sure what one party knows, that can influence better decision of the other side; is clear for both side; enhancing the partnership spirit and encourage collaboration in this industry.

In the world of Big data , is very much sad to see things going wrong due to lack of information; why are we willing to keep these data in the database, and we can not actually facilitate right information , at the right time to the right individual. Simply no hidden data , the both sides see the same information, making the decision much easier and the action of the other one is very much justified with what is there. What he did , I would have done it if I were in his shoes, no more be trailer as we know reasons why to every action.

1.2.2 Objectives

To reach to this we have 5 clear objective to support our intended mission:

1. Pre -agreed pricing

Before any production begins, buyer and farmers agree on price per unit, total volume and delivery schedule; not only these they set together the specific cash flow per defined time frame; which help them to analyse together “ are we making profit or we are lagging behind” so to easily respond to market signs earlier. This agreement is recorded in the system and visible to both parties.

This might sound as too risky for some buyer who were making fortune and they are going to be locked in by a contract; but that is the very same reason, the suppliers have been surviving on the guess game; maybe it is like this, like that. If the buyer is willing to stick with you; the collaboration grow, no need of quality check, no need of suspicious controls. And most importantly the contract bounding you, is flexible in you hands, so whenever you agree to terminate you can; and the system will record that it was mutual agreed move.

It is not being locked with some one, is to chose a trustworthy partner in the business journey, with shared interest, and common goal

2. Timestamped Shipment confirmation

When the farmer ships goods, they mark “shipment started” with a timestamped. When the buyer receives good, they mark “ shipment closed” with time stamp and record quantity received, as well the transportation details, which removes the doubt of who to hold accountable of what might happen to that package. And every single entry is on timestamp track, this services a crucial role in resolving any kind of dispute; which ensures the safe environment of operations.

3. Payment Evidence Storage

When buyer pays(using their own agreed ,convenient method of payment outside the system), they upload photo evidence of payment, the farmer confirms receipt. Both sides see the same payment status. A red “unpaid” mark appears on dashboards until confirmation.

One can claim, at this rapid development people passed this manual way; but if we picture in another direction, these people who barely understand the modern tools the manual way may seem intuitive for them, and help them to appreciate the later development of the tool; which ensures that we help our user to gradually get to use the internet to their advantage , instead of viewing it as a mysterious tool.

The more you understand the system , the trustworthy you find it, and where there is no doubt, people can find a room to focus on other profitable points.

4. Trust score Accumulation

Every completed deal increase the trust score of both parties, in the relevant contribution of each party to the completion status.This reflect the actual image of the party’s business behavior, which make sure those who are good get to get more work . On-time completion (based on agreed schedule vs. Actual timestamp) is recorded separately , and the the calculated trust scores are visible to all potential future partners.

5. Dispute Evidence Logging

When dispute occur, both parties submit, evidence. The system logs everything with timestamps. A pre-agreed arbitrator of their own choice reviews evidence and makes a decision. The decision is recorded and affects trust score of each individual.

This is not the intended thing to happen often but as people are doomed to find themselves in this case, we make sure the situation is solved internal without too much complicating court things.

1.2.3 Out of scope (idea wise)

Nobel Source has clear guide lines, in what it does and what it won’t be doing; with each a specific reason not to.

1. Process payment

Not only is risky to claim” I can handle the payment myself” as it involve people’s money from bank they trust to your newly built system; it also requires banking licenses as you will be working with those banks on the lowest level possible, which exposes a threat to their system. So going all in for handling the payment myself, assuming the cloud will trust my system; it is an increase in the

scope; totally not reasonable for my graduation project, nor it would make sense for this delicate target market.

2. Provide legal enforcement

Court system are well known to be slow and expensive. We are dealing with people not very familiar with internet, they have been working without this internet leverages. Telling them to pay a lot just to do what they can do manual. Saying it will be easier; it surely will not make sense. The approach of providing them with an environment where they can set rules and guideline that fit their situation and reasonable to meet; reducing the possibility of cheating them; and the contract is flexible between them.

3. Eliminate market crashes

Some factors are very much natural, and whatever you can do can actually work if you are on bigger enterprises, where the money cash flow is huge, and be able to omit smaller loss and smaller gains. In short; to avoid this completely is impossible, what we can only focus on is clarity in every move each side makes; this make it easy to read the signs of the market and act accordingly , without harming those around.

4. Control weather or seasons

Here it would be a lie to claim that we offer a way to control the nature. Seasons and weather are there to be predicted not controlled; what we offer is the insight of what our prediction is , in comparison on the current trend. Like we allow the room participant to set goals for their monthly transaction; and they can see when it is reducing and when it is increasing; this is a stable way to deduce the future event from the data; making a good use of the big data we always claim to have. Simply we help in facts based prediction; not a super natural promise to control the nature.

5. Replace all middlemen

This is technically impossible as some do bring value to the supply chain , and the very same internet we are providing solution on; really does not permit total removal of middlemen. In “ the diffusion of internet business and renaissance of conventional wisdom,” Malin Brannback and Jussi Puhakainen argue that while technology enables disintermediations, it simultaneously drives re-intermediation by creating a need for new digital brokers to manage consumer choice. They contend that total elimination of middlemen is unsustainable, as the market always find a way to introduce something, not labeled as middleman, but it is. For instance Nobel Source is also a sort of middleman , in a sense. So we only promise an effective communication; not a free of middleman, “disintermediation.” as this to happen we should not be in the picture as well, but we encourage that movement.

1.2.4 Success metrics

Both parties see same contract: whatever they agreed on the price, the shipment schedules, the method of payment.. and other benefit or responsibilities to be in the room, and sealed in the

1. contract, all these should be visible to both sides, in the current state; allowing proper decision taking. And clearing the communication gap right away.
2. Shipment timestamps recorded: this will ensure 100% transparency in the logistic, of our system which emphasizes on transparency instead of the exactly right cause of action. As long as the goods leave the supplier and it is marked collected and when it reached the destination it is recorded as received. That is all. the visibility is the key.
3. Payment evidence uploaded: as we are not using our own payment system, we allow to use external services that are convenient for both sides. We need to take proof of payment; this proof will be the deciding factor of whether we reached to the aim of the project.
4. Trust score calculation: Needs to reflect the working behavior of our users; in a clear familiar way, of trust score assigning. This increases the users' trust in the system as it always hints if the new partner is a good one based on his past behaviors. This to be computed after each deal.
5. Dispute evidence logged: this includes the bank payment receipt, the specific room logs, and any other made logs that are relevant to the case. This makes sure no problem is ever resulting into courts as they will be paying a lot, the very same thing Nobel source doesn't want to cause its users.

You should write down your goals and explain the purpose of the project in detail, also you need to write benefits of the project. We want to 100% of enough basis to use when resolving or de-escalating the situation.

2. Literature Survey

In the literature we see a lot of clear hints about the success of this project and what to expect as the whole process is not only straight to success. We start comparing with the current services; how they solve a bit of this problem and see how innovative Nobel source stands.

Comparison 1: Traditional Marketplace platforms

In these marketplaces we can name for example Alibaba.com and Tradekey as they are mostly widely known. For instance Alibaba.com; operates the world's largest business to business marketplace, connecting buyers and suppliers across hundreds of product categories[9]. Suppliers post listings or post products; buyers search and send inquiries. Basic messaging enables initial contact, which is totally an ideal environment for our case, of solving supplier and buyer gap. However Alibaba does not provide structured shipment confirmation beyond what sellers voluntarily upload, which clearly marks a gap in their logistic approach. Payment proof is handled through escrow services which is also a third party, as Nobel source also suggests we

won't handle payment ourselves, they will use whatever convenient , only the system require them to upload the proof of transaction", but Alibaba assumes that the negotiation and agreement happen within the messaging system ; without structured data extraction.

Nobel Source , as I said , differs by treating the post as the beginning of an event sequence, not the entire as the whole purpose of the system. Every post can evolve into a structured contract with pre-agreed terms. Shipment confirmation is timestamped and requires both parties to mark status changes. The platform does not assume trust ; it build one; through shared visibility.

Why Alibaba has not solved this problem

Alibaba's revenue model depends on supplier subscriptions and transaction fees , which is not cheap to begin with; as this case of ours deals with fragile market, where pricey things are not even given change, big industry like Alibaba , is not going to risk all for this.

Secondly for them to do the structured coordination, they would have to take sides in disputes, which conflicts with their marketplace neutrality position [10].

Simply Alibaba is good for the business to be public and showcase their production and get customer around the world; it is not mainly meant to solve the misinformation between the supplier, and buyers; in terms of collaborative and structured planning. For them they connect them so the sell and buy. That is all.

Comparision 2: Logistic Management Systems (Freightos ShipBob)

Freightos provide freight rate comparision and booking across hundreds of carries[11], this is pure logistic in short, where by Nobel source to make logistic as flexible as possible. The platform optimizes for transportation execution- finding the cheapest or fastest shipping option. However, freightos assumes that the commercial negotiation(price of goods terms of sales) has already occurred elsewhere, as they are 2 different things, money transaction and logistic, this marking a risky connection where they can easily deceive each other. The platform does not connect the commercial agreement to the shipment tracking;This make these service expensive as well for some people and make the main aim of the approach less effective.

Nobel source integrate logistics tracking with the same event stream as negotiation. The same interface that shows the agreed price and other working guide line in the contract; also shows the shipment status. This integration is critical for dispute resolution when a buyer claims " goods never arrived", the system shows both the supplier's shipment started timestamp and buyer's shipment closed time stamp or absence ; which guaranties a safe environment for business where we don't rely on assumption rather focus on what was logged into the system

Why Freightos has not solved this problem

Freightos is a logistics company, not a trade coordination company, which make a whole another direction of the business , in comparison with our case of focus. Their expertise is in moving goods, not in structuring commercial agreements between counter parties[12].To solve the problem is to target the business practices beyond only shipment and logistic and it matters to

how the other party behaves, means the way they all plan together, and planning is not only bound on logistics

Comparison 3: Messaging -first trade platform(whatsapp business, wechat)

According to Meta. Over 200 million businesses use whatsapp globally, and a significant portion of agricultural trade in africa and asia occurs through the platform[13]. Farmers share photos of produce..Buyers respond with offers. Agreements are made through conversational text, which is in text form no further structure. The advantage is zero structured data extraction, but this advantage is the bottle neck again as the no structure means you can not easily see the logs of what happened. When a dispute occurs , there is no searchable record of who agreed to what and on which date.

Nobel source preserves conversational freedom while imposing light structure, the platform provides contract templates with guided fields, but the negotiation can still occur through chat the difference is that the final agreement is captured in structured fields(price , volume, dates, that can be queried, displayed and audited. And allow to plan for future activities , much more formal than how whatsapp would do. Not only whatsapp all other messaging apps, tend to work in the same way; which is not helpfully enough to solve the supplier buyer gap. Reason why we need better approaches for better solution yield.

Why Whatsapp has not solved this problem

Whatsapp is general-purpose communication tool, reason why it is even so popular. Adding structured trade coordination would require fundamentally changing the product from a messaging app to vertical SaaS platform, which is not Meta' strategic direction[14]. And would mean to sacrifice a lot of people who enjoyed the app due to not being packed with all these business level logic. Which Meta is surely not willing or ready to go for at least for now.

Comparison 4: Enterprise Enterprise Resource Planning Modules (SAP trade Management , Oracle SCM)

System Application and product in data processing(SAP ,German, ERP); it's trade management modules provides comprehensive contract management, claim handling and rebase procesing for large management suite includes logistics, procurement, and order management. These systems solve the coordination problem completely- for organizations that can afford six-figure implementation costs and dedicated IT staff, This is totally not the case for our supplier and buyer gap; reason being, even though this is a problem , they have been surviving for this long; so if we bring is a need of this huge amount of money ; they will be will to keep suffering rather than giving a chance to the technology.

Nobel source targets the informal - to formal transition zone, smallholder farmers and small-scale buyers cannot afford SAP, they do not have IT departments, they need a system that needs internet, the platform is designed for this constraint we don't assume the mid-way of the transportation the transport will have internet, we don't expect 100% online. What we need is the right information being share as soon as possible. Taking the package we log(assuming at the collection cite they have internet) and log again once it reached to the destination) , instead of

assuming that all the time these people will be online. Which is not the case at least in africa region and in most developing countries.

Why SAP has not solved this problem

SAP's target customer is the fortune 500, not the smallholder farmer. Lowering the price; point to reach small-scale agriculture world destroy their enterprise software margins[16]. So the way SAP operates is by nature not in favor with our case of focus; we need cheap and efficient approach; but in SAP world efficiency come with price; making a product that operates on our level worthy million ; that these people are not yet willing to offer for this problem.

Comparison 5: Active agritech startups

Several startups are actively working on related problems, Wikifarmer, founded in 2020, raised \$7.7 million to build farmer-to-buyer platform focused on transparency[17]. plamshops in nigeria storefronts. Sarisuki in the Philipines aggregates farmer groups for collective purchasing. All these they share one thing; make money before all. They start the project or even the company , with the idea to generate profit as early as possible so they can pay workers and make a leaving from it. Which is not always the case as for this supplier buyer gap' if you rash for money; the resulting product will be expensive as all other tools out there and a bit more risky, as it is less exposed to the testings.

Nobel source differs in its explicit limitation of scope. These startups raise venture capital and must promise large markets and rapid scaling. Nobel source promises one thing only; shared visibility. The platform does not claim to fix logistics , finance or legal enforcement. This narrower scope is a feature ,that makes my system cheaper to build, easier to maintain and more honest in its promises.

Why these startups have not solved this problem

They want money, the problem is a business opportunity for money generation; these startups companies are in hurry to expand scope .Many agritech startups have collapsed(Otipy) because they tried to do too much. Logistics, payment , on margins that could not support the complexity[18], and at the same time in short period of time. Reason why Nobel Source refined its approach; we are not aiming for fast production that will yield the instant profit; what matters if the targeted customers(suppliers, and buyers) to understand the need of the change in the marketplace, and find the Nobel source way convenient enough and safe for them to change to it.

3. Background Information

Modern software engineering methodology priotizes developer velocity, strict type safety and the minimization of configuration boilerplate. For this project, a unified typescript ecosystem was selected to replace the old ways of doing things. Traditional web development strategies often introduce significant friction through manual database schema synchronization, complex css compilation pipelines, and fragmented client-side state management. This architecture implements next - generation tools designed to abstract lower-level infrastructure operations into automated workflows, declarative syntax, or single-line cli commands.

I avoided writing manual sql schema migration scripts, provision custom data administration portals, building repetitive user interface components from scratch, by either using the tools to be listed

bellow, of defining them once and reuse later. Hence establishing independent validation layers across network boundaries.

3.1 Technological Adoption Strategy & Learning Curve

I admit I found a lot of these new tools on youtube, while scrolling, and they have on thing in common; they responded to the problems I was having during development, maybe the tedious task to do again and again, some time involving designing mastery, which I don't have at least for now. Or others were to nice of a job; And decide to try them.

3.1.1 Context and market research

The choice of this software stack was driven by continous observation of current , market trends, software engineering forms , and developer podcasts. In web development, tool fragmentation often leads to maintenance fatigue. To prevent this, the selection criteria targeted utilities that explicitly solve common integration issues rather than just relying on established legacy options. When a tool demonstrated a proven ability to remove systemic friction(such as manual configuration, most of it being tailwindcss; as am weak in css but a huge fun of tailwind. But whenever I set it up , the blue hello world word t render never became blue.” simply it never worked , unless I use a template and build on top of it.).

3.1.2 Technical Adaptation and mastery statement

The software dependencies used in this application represent the current state of modern web architecture. Several core frameworks- specifically React 19, Vite 7, TailwindCss@4 , Drizzle ORM(Have been recently released or undergone major architectural re-engineering, I don't know but most of the things are not youtube wise documented , I relied on one video , from drizzle themselves; so this marking a key disclaimer, me understanding every bit of the underlying logic , it would be a lie only at least to what I used, but for the whole architecture , it require reading the documentation very well, thing I did not do).

NB: This project does not claim 100% theoretical mastery of every internal framework abstraction, instead , it demonstrates functional implementation, successful integration , and practical problem-solving using current documentation/ mostly tutorials. Transitioning from manual boilerplate setups to an automated development pipeline reduced repetitive tasks and configuration overhead by approximately 40%, directly speeding up feature delivery.

3.2 Required & Used Software

● PostgreSQL 18.2 :

PostgreSQL serves as the primary relational data store, selected for its reliability, strict compliance and native handling of complex data structures. And I was in for drizzle as I saw it having cool feature and postgresql was very compatible, and it uses the normal sql language, so not much form me to re-learn; and any difficulty I was to meet , reading the documentation , it was gone be clear. Due to my abit not strong database design; I needed a databased that can support jsonb.

- **JSONB Document Support:** the application uses an encoding -first design philosophy, embedding structured metadata inside JSON columns (JSONB). This approach allows the system to store changing feature parameters without needing frequent disruptive schema alterations or a proliferation of single-purpose relational tables.
- **Timezone-Aware Temporal processing:** the database enforces robust timestamp handling with explicit time zone support(timestamp with time zone). This prediction is mandatory for establishing legally auditability record of shipment arrivals verifying multi-party custody handoff, and calculating contractual dispute windows.
- **Resource efficiency:** PostgreSQL maintains high operational throughput under constrained conditions, allowing the platform to run efficiently on the entry-level cloud infrastructure.

● **Node.js 22.19.0 & Express 5.2.1 :**

The backend architecture relies on an asynchronous runtime environment designed for rapid concurrent request processing..

- **Non-blocking I/O architecture:** Node.js manages incoming network requests using single threaded event loop, this approach allows the backend to efficiently handle concurrent connection from multiple farmers logistics providers and commercial buyers monitoring real-time shipment events.
- **Express framework:** the API routing layer implements the newest version of Express, this provides a lightweight framework for handling RESTful endpoints, avoid the unnecessary abstractions, strict architectural rules, and heavy startup times of enterprise framework.
- **Native ECMAScript Modules (ESM) :** the backend project configuration specifies type: module. This enables native support for modern javascript import and export statements, eliminating legacy commonJS compilation steps and standardizing the codebase with the frontend repository.

● **React 19 & Vite 7:**

The client-side architecture moves away from legacy frameworks to leverage an optimized, modern runtime engine.

- **React 19 Component Framework:** The application uses the latest major version of React to manage view layer rendering. This component-driven architecture ensures user interface elements- such as supplier posts, live transit trackers and customer trust badges- are modular and highly reusable across different application layouts.

- Vite 7 Asset Compilation Engine: Vite 7 replace legacy webpack tooling. By utilizing native browser ESM for local development and rollup for production bundling, vite enables nearly instant local server starts and extremely fast hot module replacement during debugging.

● TypeScript 5.9 :

TypeScript acts as a static typing layer across the entire codebase, enforcing consistent contract enforcement on both sides of the network boundary.

- Compile-Time verification: Typescript analyses application source files prior to deployment , catching structural mismatches, underfined objectm and syntax errors before the code reaches a staging or production environment.
- Data Layout Synchronization: typescript ensures that complex JSON payloads, transport state variables and transaction records , match their expected formats throughout the aplication, preventing runtime, exceptions caused by unexpected data forms.
- ESLint: along with Eslint by restricting the any type , by default; it helped me to have a consistent well types object in my entire project, . there are some file though , I had to disable the any type, warnings, as I used the generic function or the resulting types were not of clear structure, instead of type1 |type2 | .. going for a long list, I chose to leave it as any.

● Drizzle ORM 45 & Drizzle Kit

Drizzle ORM replaces traditional Object-relational Mappers by providing full type-safety while preserving standard sql syntax. The application configuration utilizes two key automated scripts.

- Automated schema push : under the script of “npm run push” I was executing “ drizzle-kit push “. This script examines the typescript schema file. Schema.ts and directly modifies the remote postgresql database layout to match . this eliminates the tradictional process of manually drafting eoor-prone sequential sql migration files.
- Automated seeding: under the script “ npm run seed:db “ I was running “tsx seed.ts” where this seed runs on the edge ; by using the database instance and the schema, types to insert dummy data into the database for production . this running as script , speed up the process and anything that changes in the schema we immediately see the types mismatch in the seed , hence seeding correctly.
- Database Administration studio: by running script “ npm run db: studio” , I was executing “ drizzle-kit studio”; running this command launches an interactive, web-based databases management interface. This built-in utility serves as an immediate , functional administration control panel, removing the need to spend development time coding a separate admin interface.

● Shaden CLI & TailwindCss 4.0

The application styling pipelines relies on automated tools to build a responsive , cohesive user interface.

- Tailwindcss v4 compiler Engine: the application uses the newly designed `@tailwindcss/vite` plugin. This engine compiles style utilities directly inside the vite build process. It functions with zero separate configuration files, zero postcss plugging and zero manual style sheet management, reducing system setup to a single package import.
- Shadcn UI component Automatic : rather than installing large inflexible packages, the project uses the shadcn cli tool ([shadcn@4.2](#)) . this tool injects accessible component source code directly into the local repository. This gives the developer full control to customize elements like buttons, dialog boxes, and forms without having to build accessible, keyboard-navigable UI primitives from scratch.

● Tanstack React query V5

Managing remote server data inside react components historically requires complex combinations of useEffect hooks and local management.

- Declarative data fetching: react query replace custom custom fetching logic with automated hooks that manage the asynchronous lifecycle of API calls.
- Automated Cache Management: the library automatically handles caching, data garbage collection, network re-fetching when a user focuses on window, and loading states, providing standard feedback elements to the end user.

● cloudinary & multer AUtomed media pipeline

Along with using ORM, this is the second trick I learnt from my internship, the database we usually use in class, MySQL, postgres... they are not designed to store media, we use “buckets” third party storage which are design and optimised in keeping and serve media data. In which we have cloudflare (it is super good but requires credit card to access it), cloudinary (completely free, and if you pass your free plan it stop work, unlike others that will start deducting money on the credit card you provided).

- Multi-part Request resolution: Multer process inbound http multipart/form-data uploads. They handle file streams cleanly , prevent server memory saturation by managing binary fragments on the disk during file operations.And this is the only way I know , about processing files in the api architecture.
- Cloud Asset management : the cloudinary SDK process uploaded files, offloading image optimization, compression and global content delivery network (CDN) hosting to external cloud storage. It returns secure immutable URLs to store within the database(postgresSQL).

3.3 Graphics & runtime tools

I also use tools to optimize my coding experience on the runtime behaviors of my code, and when I had to design or make any change to the image I was to use ; which did not happen a lot, but I couldn't rely on AI; it is slow to grasp the logic I have with in.

- **Figma** : I used it as the primary design workspace, to create the platform's logo which happen to be a lot faster and straight forward. Hence saving me a bit of time.
- **Shadcn create project**: link [<https://ui.shadcn.com/create>] .Here is was very much like a cheat on the ui designning phase; I did not really design much; the last time I designed my ui; it came out very bad, so this time around I was not going to risk it again. Here you just toggle between different presents in typography , primary colors, icons library and all you have to do in going in the terminal and say: "npx shadcn@<version, defaulted to latest> init --preset <preset code you saved from the website>". And that is all, you have your project, color set, tailwindcss in, them toggle global context set, only need to add routing system and you start coding.
- **Bcrypt & Express-session**: because I had to create the id myself not to rely on the database auto- increments, plus I had to create the very same unique ids for my session token in express-session. Which helped to meet the database constraint of uniqueness on the id column. I did not use (uuid) which is the global standard for unique safe , and database generate unique ids, reason is that I don't understand how it works, and they tend to generate very long ids.
- **React Router DOM v7**: manages client side navigation , page transition and structured layout engines, simply the routing system we usually see with in websites, we design them using this.
- **Recharts**: I saw this recently on the youtube and as I was to need bar chart, I used it to learn how it works even. It is a composable charting library built on React components, used to display graphical charts and system analytics without requiring direct SVG manipulation.
- **Framer Motion /Motion**: this is the heart of all my motions in the whole project. Motion allow use to create that liveliness of the website , by responding to the user interaction by nature like motion of the components, they interact with. For convenience ; I created a separate file . ts and defined all my mostly to be used variant of the motion as it is just a definition of (initial state, duration , ..) by changing these we get a whole new motion, for the uniformity of my project , I was reusing the very save variant , in my motion divs.

3.4 Development tools

- **Mercurial**: as instructed , this was the version controller to use. Using Mercurial it was a bit straight forward than git , as now I understand a bit of version control; which even couple of years on git , I could not understand.
- **VS code**: This is my number 1 got to IDE , when it come to javascript or typescript; cause it is a very familiar interface, more than any other editor in my pc. And above all I have pretty much extension in there that make my coding experience.

3.5 Hardware Environment profiles

1. Developer environment

Nothing much , required , only a computer that can access internet will do; my ststem is as follows:

Brand: Lenovo

System type: x64-based PC

OS version: windows 11 pro

Processor: Intel(R) core (TM i7-8650U CPU @ 1.90GHZ

RAM: 16 GB

Graphic card: 128 GB

Storage: 477GB

PLUS postgresql installed locally to the pc.

2. User Hardware

Any standard consumer electronic device that can access internet and has screen and can visit websites. As now the project is a website, just accessing website will be enough, later when it goes to being a mobile phone the threshold of what to require will still be maintained to minimal, so people can easily enjoy the product's benefit without an over kill, of buying stronger devices.

4. Design Documents

4.1 Data flow diagram - Level 0 (context diagram)

The context diagram shows Nobel source Hub as single process interacting with three entities: Supplier (farmer) , buyer and system Administrator.

High level over view:

We have mainly 3 actor: Admin , Supplier , Buyer . where most of the activities are held; and we have the automated audit , that system will take care of, triggered on selected mutation to the database records. Admin there to resolve the tangled cases.

- we have option transportation; this is because there is always this , thing of transport as a must, but some business literary they can handle transporting goods themselves, as we are target small business even, so giving them a flexible transportation mechanism would attract them, and not removing the transportation totally, shows our acknowledgement to the current system in their useful side.

- admin set the trusted institution as arbitrary ; who can view the cases and guide to the calm resolution ,! this is not full well developed , but it is a good thing, avoiding to escalation of the situation, so it won't end up in the court where everyone has a thing or two, that they will loose.

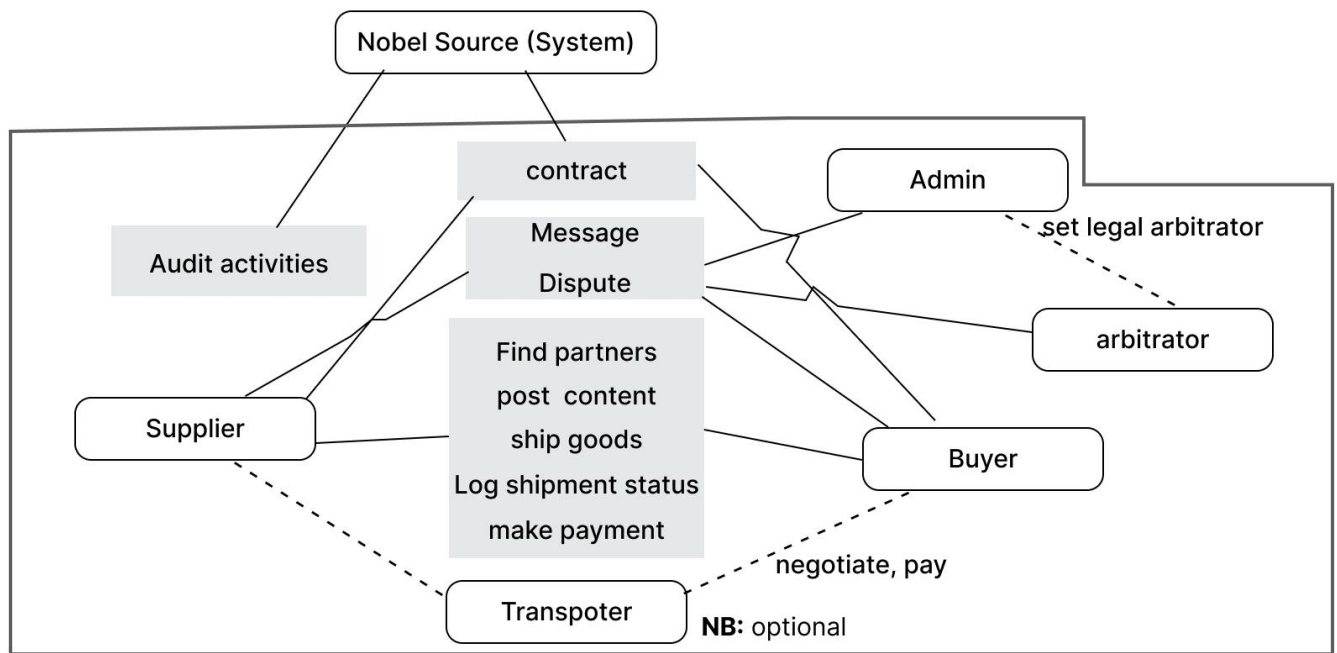


Figure 1: System over view [Nobl Source]

Formal Data flow diagram(detailed):

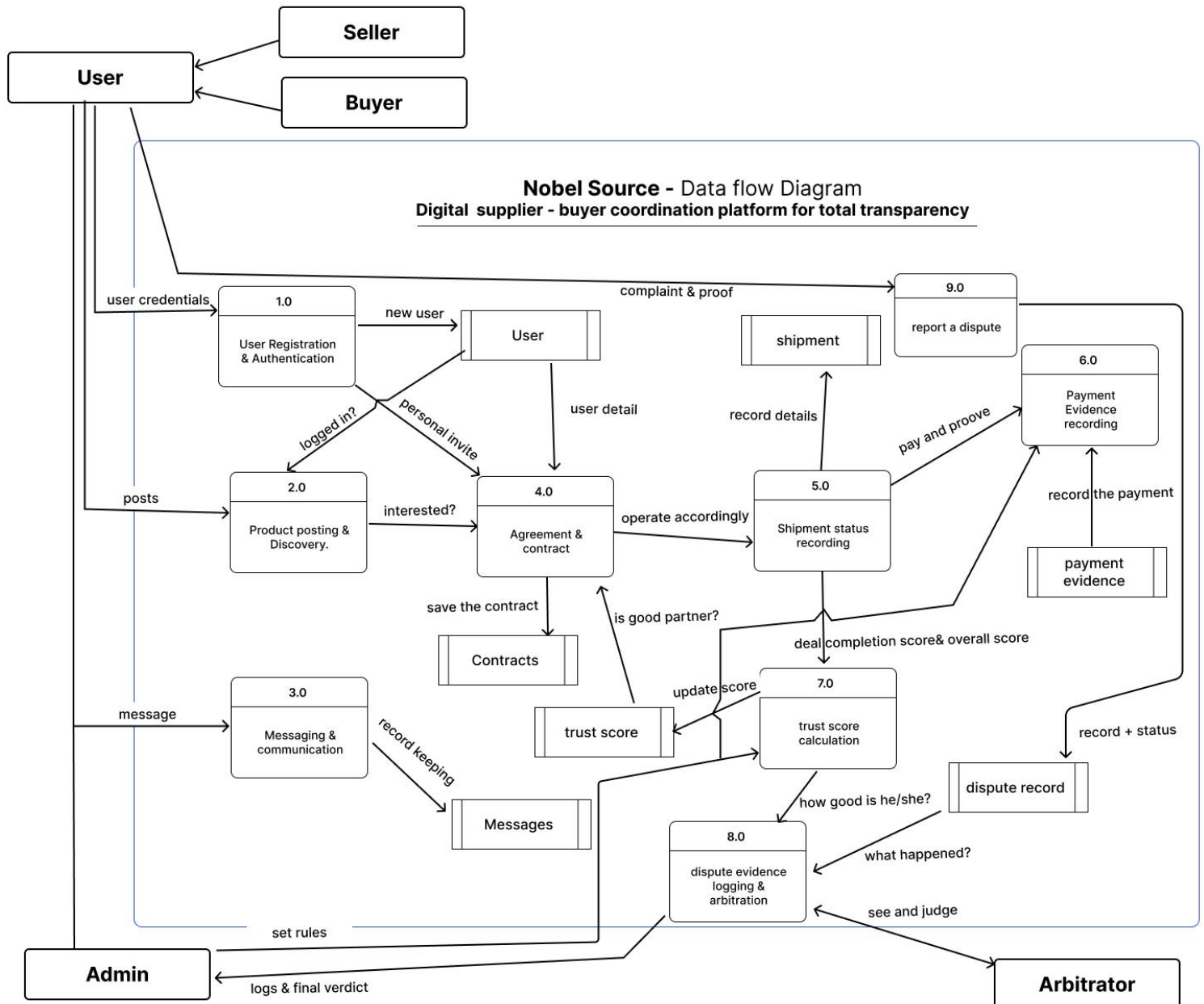


Figure 1: Data flow diagram[Nobl Source]

External Entities explained

- **Supplier/seller** : Creates supply post indicating what produce is available, marks :” shipment started” when goods leave; confirms payment receipt after buyer uploads proof, optionally chose the transporter upon agreement with the buyer; can a dispute.
- **Buyer**: creates demand posts or responds to supply posts. Marks” shipment closed” when goods arrive and are inspected. Uploads payment evidence (photo of the receipt, transfer confirmation), having a sayin in chosing the transportation method; can raise a dispute.

- **System Administrator:** Manages user accounts, reviews dispute evidence, applies administrative actions (warnings, suspensions) when platform rules are violated.
- **System:** Audit is done automatically, just some post action actions, deletes, patches , maybe later on ; the confidention “get” ; they trigger also an audit . which help in knowing what happen , when and by who.
- **Transporter(optional)** : This is literally the main reason of the whole project, we need and have to limit the exposure of these people in the supply chain; that is why I said optional , marking the option of having goods from the supplier to the buyer, zero commision , just pure communication.end to end.

4.2 Nobel Source -Use case Diagram

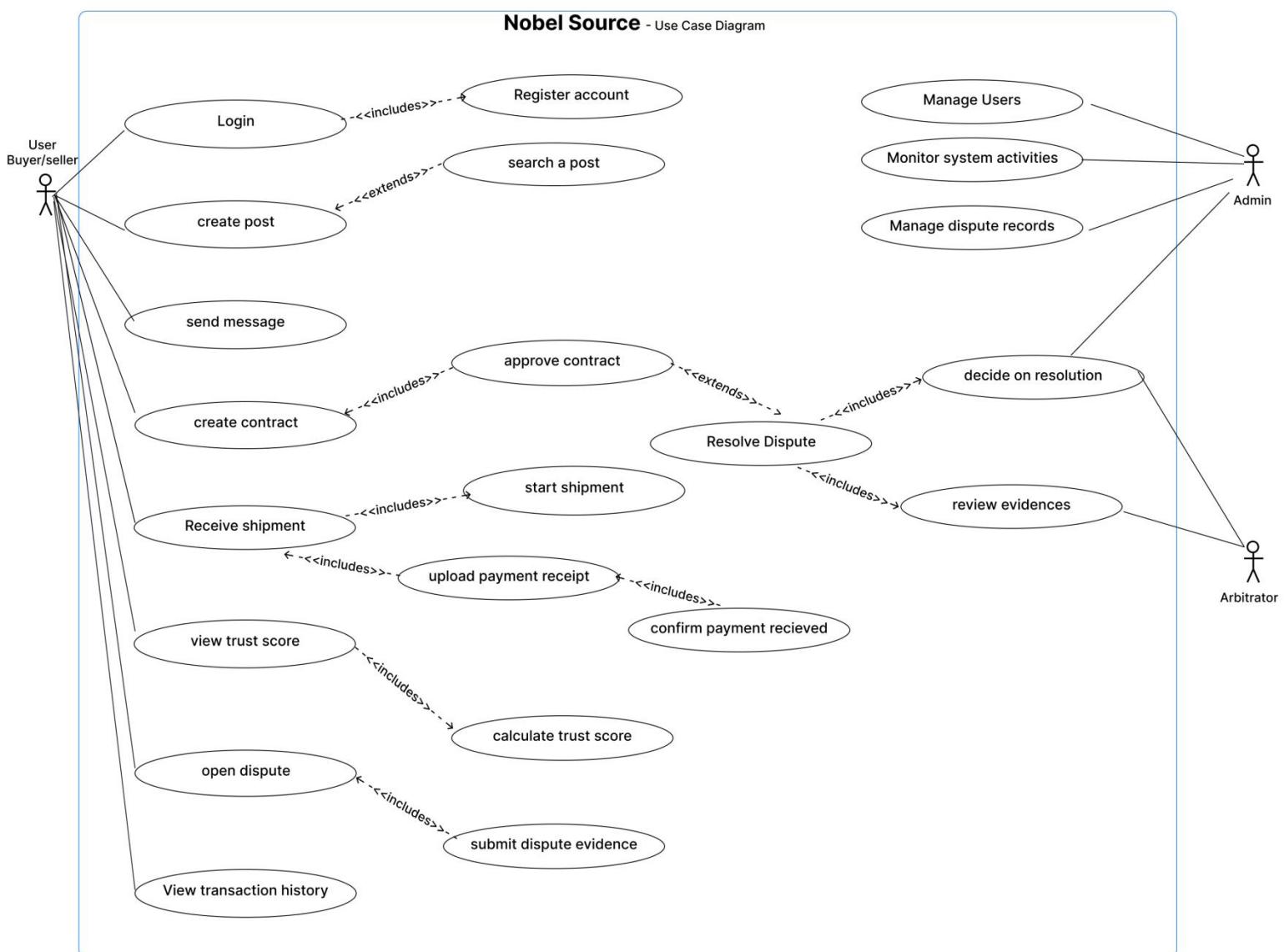
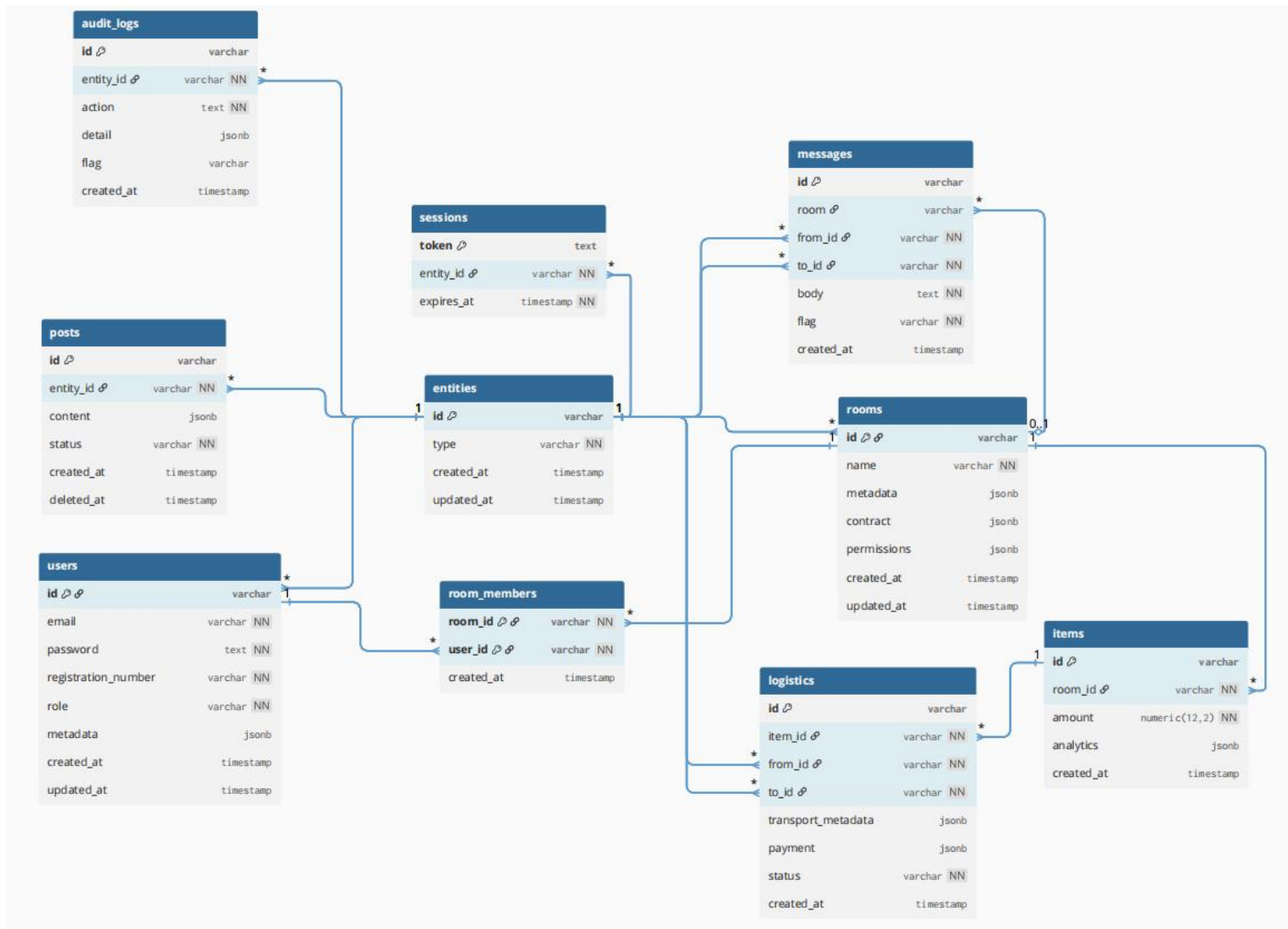


Figure 2: Use Case diagram[Nobl Source]

4.3 Nobel Source -ER Diagram (10 tables)

The database schema is defined in “backend/src/database/schema.ts” using Drizzle ORM . the core tables are as follow.

The database schema is defined in “backend/src/database/schema.ts” using Drizzle ORM . the core tables are as follow.



Entities; Users; Rooms; Room members; Posts; Messages; Logistics; Items; Sessions; Audit-logs

4.3.1 ER Diagram explained

Things were intentionally kept minimal and flexible.

Durin development, many new ideas features kept appearing. If every detail had been modeled as strict SQL cols, the schema would have been changed constantly. Migration would become difficult, and queries depending on fixed columns could easily break whenever structers changed.

Because of that, many dynamic structures were moved into jsonb fields. This is risky but good thing is that I had typescript , strictly typing everything so that , i can force discipline in my development journey.

This approach helped to:

- ✓ Reduce schema complexity

- ✓ Avoid creating too many tables
- ✓ Keep the project manageable for a graduation project
- ✓ Allow the system to evolve without constantly re writing migrations
- ✓ And leave room for future optimization , and cleanings later.

Initial the system had around 18 tables, which became unnecessarily large for the current stage of the project. The schema was later simplified so the core workflow could remain stable first before deeper optimisation and normization.

4.3.2 Core Design Idea

The whole system is center around entities table.

At first, users and rooms were treated separately, but later it became clear that most system interaction were happening between generic actors:

- Sending messages,
- Owning posts
- Participating in logistics
- Generating audits
- Authenticating sessions.

Instead of creating separate references everywhere for both users and rooms, shared entity abstraction was introduced.

An entity can currently be: **User** or **Room**

NB: The entity_id is exactly the same ID as the related table of reference.

Example:

- User_xxxx // this is a user that exist both in the entity and user table
- Rm_xxxx // exists in both entity and room table are a specific room.

This where I learnt that keep changing the business idea back fires. I realised late that this entity has little of effect, but I couldn't not change as I was almost done, I was never going to finish, so I decide to go the way it is.; but on the other hand, this leaves a room for future expansion where the platform may later introduce new entity types without changing the whole relational structure.

4.3.2 Tables

1. entities

Central abstraction table.

Used so the entire platform can reference a single generic actor instead of separately handling users and rooms everywhere.

Purpose

- Unify reference,
- Simplify messaging and ownership
- Prepare for future entity types,
- Centralize authentication and actions

Fields

Field	Purpose
id	The shared ID
type	Which table references from
Created at	Creation timestamp
Updated at	Track last update

2. users

Store actual user accounts.

Authentication and personal profile information live here.

Purpose

- authentication
- User identity
- Verification
- User metadata

Fields

Field	Purpose
id	Unique user id
email	Login email
password	Hashed password
Registration_number	Confirm Organisation/business legal exitance rights.
role	System-level permission role
metadata	Flexible user profile data.

Metadata: User_meta

Contains:

- ◆ Profile data
 - Phone
 - Name !!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!
 - Image
 - currency
 - Website
 - Country
- ◆ Rating
- ◆ Permissions

3. rooms

Represents collaborative spaces or operational groups

A room may represent a business relation, where all business decision and tracking happens, aiming for direct communication and total transparency.

Purpose

- Group Colaboration
- Contracts
- Operation coordination
- Increase one's trust score.

Fields

Field	Purpose
id	Unique room id
name	Room name
metadata	Room details and operational data
contract	Agreement structure
permissions	Room-level permission

Metadata

Contains:

- ◆ Description
- ◆ Tags
- ◆ Member rules [actor , role , isAdmin]

- ◆ Goals[monthly_target , currency]

Contract

Contains:

- ◆ Version
- ◆ Introduction
- ◆ Sections [shipment_rules,payment_terms,penalties,dispute_resolution]
- ◆ Metadata [provider_id, reciever_id, jurisdiction]

4. Room_members[weak]

Join table users to rooms

Purpose

- membership tracking
- Room participation
- Access control

Fields

Field	Purpose
Room_id	Targeted room
user_id	Participating user

Composite primary key prevents duplicate memberships.

5. posts

Marketplace or communication posts

Used for:

- Offers
- Requests
- Announcements
- Information sharing.

Purpose : act as the main public interaction layer.

Fields

Field	Purpose
-------	---------

Entity_id	Owner of the post
content	Full post data
status	Lifecycle state
Deleted_at	For soft delete records

Content

Contains:

- ◆ Title
- ◆ Description
- ◆ Media
- ◆ Pricing
- ◆ Location
- ◆ Tags
- ◆ Categories
- ◆ Post type

6. items

Represents transactional or operational items handled inside rooms.

Item represents the good that a certain room is operating, on food, raw material..

Purpose : core operational tracking object.

Fields

Field	Purpose
id	The unique identifier
Room_id	Room of origin
amount	Financial or measurable value
analytics	Operational item data

analytics

Contains:

- ◆ Inventory information
- ◆ Sales statistics
- ◆ Demand analysis

◆ Participants

◆ Operational state

This is my main motive to make things very dynamic, I reach at a frontend page , and found, Oh I need to know this on the item, and I find that I don't even have it in the database, I had to come and add it here, and seed with it. And that is all , am back on track , not to much of work.

7. message

Communication system between entities.

Supports:

- direct communication
- Room communication
- Proposals
- Notifications,
- System messages.

Purpose : unified communication layer.

Fields

Field	Purpose
room	Optional, like affiliation
From_id	Sender entity
To_id	Receiver entity
body	Message content
flag	Message category

8. logistics

Shipment and transportation tracking.

Handles movement of items between parties

Purpose :

- Shipment coordination
- Payment tracking
- Transport information
- Delivery lifecycle

Fields

Field	Purpose
Item_id	Related operation item
From_id	sender
To_id	receiver
Transport_metadata	Transport details
payment	Payment information
status	Shipment lifecycle

Transport

Contains:

- ◆ Transporter type[solo, company]
- ◆ name
- ◆ Phone
- ◆ Vehicle plate number

Payment

Contains:

- ◆ Amount
- ◆ Currency
- ◆ Proof confirmed_by
- ◆ Account_number
- ◆ Method_of_paymnet

9. sessions

Authentication session storage

My own managed session , I was not understanding the one react provide, when I watched youtube the all explain something like a volatily table like, where we push a sesion record and on its end the record is delete. So implemented it myself; but un optimised.

Purpose :

- Login persistence
- Token managemnt
- Authentication lifecycle,

Fields

Field	Purpose
Entity_id	Authenticated entity
token	Session token
Expires_at	Expiration timestamp

10. Audit_logs

Tracks important system activity

Used for:

- monitoring
- traceability
- Accountability
- Security analysis.

Purpose : System transparency and operational history.

Fields

Field	Purpose
Entity_id	Actor responsible
action	Performed action
deatil	Additional event data
flag	Severity level

5. Methodology

This section presents the system methodology as both an implementation record and a design rationale. It combines the actual development steps (backend, database ,API,frontend) with the reasoning behind architectural decisions, including trade-offs made under time and requirement constraints.

5. 1 Development Context and engineering approach

The system is developed in a graduation-project environment, where requirements were not fully fixed at the beginning and evolved during implementation. This influenced both technical decision and architectural direction.

Instead of following a strict waterfall process (design - implement - deploy) the system was built using:

- Incremental feature development
- Continuous schema evolution

- Repeated restructuring of backend components based on observed behavior.

Core constraint model

- Limited development time
- Changing feature scope during implementation
- Incomplete initial domain definition
- Need for working system early (demo driven development)

This created a system optimized for adaptability rather than complete correctness.

5. 2 Database Environment setup and initialization

The backend was built using local PostgreSQL instance configured for isolated development. A dedicated database user was created to prevent reliance on system-level administrative access.

Code:

```
# create user shema22243402 with password 'mypassword';  
# Create database nobelsourcedb owner shema22243402  
# Grant all privileges on database nobelsourcedb to shema22243402
```

Engineering rationale

The decision to grant elevated privileges during development was intentional:

- Schema changes were frequent
- Migration stability was not guaranteed early on
- Strict production-level restrictions would slow iteration

5. 3 Schema Design Evolution

The initial schema started as a traditional relational structure:

- Users
- Rooms
- Post
- Messages

However , as development progressed, additional requirement emerged:

- Logistics tracking
- Audit logging
- Payments
- Item analytics
- Session context switching

This led to rapid schema expansion, which caused:

- Migration conflicts
- Enum inconsistencies
- Relational over-normalization
- Frequent structural breaking changes

5.3.1 Transition to JSONB- based flexibility layer

Instead of continuously adding new tables, a hybrid approach was adopted

- Relational core for stable entities
- JSONB for evolving or experimental structures

Example: Post content stored as JSONB

```
interface content {  
  title: string;  
  body: string;  
  media?: string[];  
  price: number;  
}
```

Justification

This approach was used because:

- Feature definitions were still evolving
- Schema migrations were becoming expensive
- Experimentation required fast structural changes
- Strict normalization slowed development progress

Trade-off

- Yes it had fast iteration
- Yes it reduced the migration complexity
- But reduced strict relational integrity
- And requires runtime validation discipline.

Technique favored development velocity not schema purity.

5.4 Schema Rest Strategy

Due to frequent schema evolution , migration drift became a recurring issue:

- Enum mismatches
- Partial schema updates
- Orphaned tables
- Broken relations

To solve this, a full reset pipeline was introduced. Under few rule.

- No table insertion to be done manual, as the next database wipe out it will clear it without a backup, everything had to be done through, the script.
- Changing anything in th schema, you also change the seed as well, so the main focus is to seed what is correct.

Developemrnt scripts

```
"scripts": {  
  "dev": "tsx watch src/server.ts",  
  "db:wipe": "tsx src/database/nuclearReset.ts",  
  "db:push": "npx drizzle-kit push",  
  "db:seed": "tsx src/database/seed.ts",  
  "db:studio": "npx drizzle-kit studio",  
  "seed:force": "npm run db:wipe && npm run db:push && npm run db:seed"  
},
```

So whenever I seeded “ npm run seed:force “ I was sure nothing is going to break, which gave me peace of mind.

Instead of repairing incremental migration failure, the system was treated as , fully rebuildable infrastructure during development.

This ensured:

- Deterministic schema state
- Reproducible testing environments

- Elimination of migration complexity overhead

5.5 Entity-centered Architecture

A key architectural decision was the introduction of a centralized identity system through the entities table.

Concept

Instead of separate foreign keys:

- user_is

- room_id

The system uses “entity_id”

Good thing is that this entity_id we can still know if it is user or room based on the prefixes,

USR_ is for users ; and **RM_** is for a room

Justification:

This abstraction was introduced because multiple modules required cross-entity interaction:

- Messages(user- user, user - room)
- Audit logs(user and room action)
- Posts(owned by any entity type)
- Sessions (content switching between user and room)

Without this abstraction:

- Duplicate logic would exist across modules
- Queries would require multiple conditional joins
- Schema complexity would scale linearly with features

Trade-off

- Unified interaction model
- Reduced duplication
- Polymorphic foreign key limitations
- Requires application-level discipline

5. 6 Backend Architecture

We start from connecting the backend to the database, as that is the whole idea, communicate with your database and serve the data in API end points.

5. 6.1 database connection

Pre-requisite: Added my database redential in the .env file ; this was to provide one source of true, and secure them as we don't push .env ; which is the standard way of keeping those precious credentials.

At least the postgresQL , need user name , host name , password and port which is 5432 by default, and if you find it is occupied ; it is just means there is another instance running you may need to find them , teminal them(kill) , to free the port, and that is all. Not confirming whether the port is available. Will later create issues when connecting database to your backend , for sure; and if you are not familiar with what is happening , this may become hours to days of debugging, a miss configuration thing

1. Check if the port is free

```
-----
PS C:\Users\Shema> netstat -ano | findstr :5432
TCP  0.0.0.0:5432      0.0.0.0:0          LISTENING      6464
TCP  [::]:5432          [::]:0             LISTENING      6464
-----
```

The in take here is ; it is a tcp network protocol, listening to all addresses , not external network connected to it and is currently listening of active and the process id is **6464**.

So to know the exact name; of this PID related process; which is very important , cause we should never kill any process unless you truly knows what it is.

2. Check the name of the PID

```
-----
PS C:\Users\Shema> get-process -id 6464

Handles NPM(K) PM(K) WS(K) CPU(s) Id SI ProcessName
-----
521 14 3540 4080 6464 0 postgres
-----
```

Most of these I don't know them , but what is clear is that postgres is the one running on this, so you either know what it is helping you with , if you doubt check online, even ask AI, know what that program is helping you , if it nothing much then , and only after that try to terminate it.

3. Terminate the process

```
-----
PS C:\Users\Shema> tasklist /FI "PID eq 6465"
```

INFO: No tasks are running which match the specified criteria.

PS C:\Users\Shema>

So this was to destroy the process but I made sure to put the other one , as destroying a process that is it, you should be vigilant.

So now the port is free we try to login in our account. First.

PS C:\Users\Shema> **psql -U shema22243402 -d nobelsourcedb -h localhost**

Password for user shema22243402:

psql (18.2)

WARNING: Console code page (437) differs from Windows code page (1252)

8-bit characters might not work correctly. See psql reference

page "Notes for Windows users" for details.

Type "help" for help.

nobelsourcedb=>

After having the user, database, host , port. We go in backend to we connect thing.

env save those credential:

```
DB_USER= shema22243402
DB_HOST=localhost
DB_NAME=nobelsourcedb
DB_PASSWORD=XXXXXXXXXXXXX
DB_PORT=5432
```

4. Connect to database

We need the :

➤ "pg" library the engine to connect the 2 worlds (script and postgres database)

- Drizzle: Object relation mapper : help us to write none raw sql code, and use typescript codes, marking the beginning of the transition.
- And dotenv for you to load the .env data into your script

Step:

1. Create a pool from pg
2. Initiate the object with credentials
3. Prepaire the drizzlelised pool ; which needs the schema details, as well and the raw pool to run when seeding, as it will be runned out side the server engine, not in the run time of your server.

```
const {Pool} = pg
```

```
const pool = new Pool({
  user:process.env.DB_USER,
  host:process.env.DB_HOST,
  database:process.env.DB_NAME,
  password:process.env.DB_PASSWORD,
  port: process.env.DB_PORT ? parseInt(process.env.DB_PORT) : 5432,
});
```

```
const db = drizzle(pool,{schema});
export {db,pool}
```

5. 6.2 design schema

Here we literally need only the core module , to help you create tables, constraints, indexes .. to build you tables; **drizzle-rom/pg-core**

The approach for the schema file was quite unique but I found it very help full.

Everything to deal with schema , is in one file ; around 430 lines ; which is the worst practice to keep those huge things together ,but with better disipline it is the best.

Structure of the file is .

1. IDs prefixes

```
export const IDs = {
  user: "USR_",
  room: "RM_",
  room_member: "RMME_",
  post: "POST_",
  item: "ITEM_",
  message: "MSG_",
  logistics: "SHIP_",
  session: "SESS_",
  audit: "AUDIT_"
} as const;

export type IdTypes = keyof typeof IDs;
```

This is Object based enumeration , combined with typeof; what I did ; I force typescript to stop seeing “usr_” as a generic string type but as a literal “usr_” by making it as **const** saying. This is the only possible format , and it is constant it never will change, so this object is like this forever.

Then we extract the types of keys in this object we make the general type of it.

2. Enums

These are normal enumerations that are directly to be used in tables, they usually are simple , with few options.

```
export const user_role = pgEnum("user_role", ["user", "admin"]);
```

So pgEnum is from the pg-core , it describes the “**user_role**” enum as **user** or **admin**.

3. Const types

Using the Union types

```
export type AuditFlag =
  | 'info'
  | 'critical'
  | 'security_breach';
```

Literal type literal , the straight forwards , by just listing the exact value we expect this type to allow.

Or we use Array to type approach ,

```
export const currencies = ["USD", "RWF", "EUR"] as const;
export type Currency = (typeof currencies)[number];
```

Which is nearly like object but here, is just values in a array form, we make the constant, so they are recognized as literal, then changed into types. Why to use [number]; we assign every index to its respective type; if we had no number; then the type currency should be of length 3 exactly as a must and it is as read only, but as number we give them option, either match index 0,1,2 ... or not.

NB:

- **object based type:** Self constructive value map; good if you want to type the keys and values at once. They are iterable at run time. Nothing can change this is more of a contract than being a type.

```
Object.keys(IDs).forEach(key => console.log(key)); // "user", "room"
Object.values(IDs).forEach(val => console.log(val)); // "USR_", "RM_"
```

- **Array based type:** the source of truth for iteration, iterable at runtime as well, unlike the contract like, you provide the value and it is validated on this array based type.

```
currencies.forEach(key => console.log(key)); // "USD", "RWF", "EUR"
```

- **Pure Union type :** more like gohst types, or the usual known types of string, number ... ; they are not iterable at runtime, as it is not even an array, you simply gave choice the compiler, to allow thing with data type if they are one of the enum value. Which it the whole intention of enum, but later on you can not extract those values to serve them.

4. Json Types

This is just a json typed, and any object given this type should come in exactly thing format, that is it. Nothing else to bargain on, no choices given here.

These are usually used to combine various above discussed types, and use the normal simple ones, like number, string, boolean..

5. Tables

This is the main purpose of the schema file, others are the usual typescript usefull nemesis “types”.

```
export const entities = pgTable("entities", {
  id: varchar("id", { length: 30 }).primaryKey(),
  type: entity_type_enum("type").notNull(),
  created_at: timestamp("created_at").defaultNow(),
```

```
updated_at: timestamp("updated_at").defaultNow()
});
```

Nearly like norm sql ;

Syntax:

```
export const [variable name] = pgTable( "[database table name]", {
  [column_name]: [data type]("[database column name"], {[options]})
    .[modifiers]()
    .reference(() => [Table].[column], {onDelete: "[Action]"}),
},
(t) => [
  [Extra constraints type]({ [constraint options] })
]);
```

Explained:

- **Variable name:** any name to use in calling the table in the javascript ; you give any name you like, but better related to the table.
- **Database table name:** any name as well, better similar to the variable name to avoid confusion, but this name is effective in the database.
- **Column name:** these are the column name you want to create inside your table. They affect both in javascript side
- **Data type:** these are the drizzle types that reflect to what type you intend to keep in here; varchar, text, timestamp, jsonb, numeric ... , pgEnum , defined in the enums.
- **Database column name:** these should be the same as the column name , so not to confuse you ; but these take effect in the database itself.
- **Options:** these are javascript configuration details, like {length: 30} , {precision: 12, scale: 2}... just to be specific of the type of the column
- **Modifiers:** from drizzle that translates to the constraints, the most used ones: primaryKey(), .notNull() , unique(), .\$Type<T>(....), defaultNow()
- **Table.column :** link back to another table
- **Action:** the behavior of this particular process; like cascade, restrict, set null or no action
- **Typescript interface:** a typescript type, like : <user_metadata> , <string[]>

- Extra constraint type: this is when you want to add constraint on multiple rows; the normal constraints; like primaryKey , unique. But indexing is only done here.

6. Table types

Then we need to extract the types of these table for insertion (to use in the seed file) and the select type (to use in the services..to be explained later).

```
export type Entity = typeof entities.$inferSelect;
```

```
export type Entity_in = typeof entities.$inferInsert;
```

5. 6.2 Seed your database

So this file will run outside the server engine ; so we need the db instance of our pool ; bcrypt to create our unique ids, and all types in schema

```
import { db } from "../db";
import * as schema from "../schema";
import bcrypt from "bcrypt";
```

In the main function of this seed; we first clean the database, by deleting table in the reverse order.

```
await db.delete(schema.audit_logs);
```

Audit-logs, sessions,logistics,messages,items,posts,room_members,rooms,users,entities

We prepare the ids to use and the password.

```
const IDs = schema.IDs;

const hash = await bcrypt.hash("pass12345", 10);
```

```
// ids
const adminId = `${IDs.user}ADMIN001`;
```

So how we seed, we create an array of entries to the database, and type guard so we avoid errors, in early stages

```
const EntityData: schema.Entity_in[] = [
  { id: adminId, type: "user" },
```

```

{ id: farmerId, type: "user" },
{ id: buyerId, type: "user" },
{ id: transporterId, type: "user" },

{ id: agroRoomId, type: "room" },
{ id: logisticsRoomId, type: "room" }
];

```

We use **Entity_in** as those are insert type ; the only difference between insert type and select type is that insert types when things can be null, they also expect it to be undefined, simply you don't provide it at all, which is true, in order to have that entry in the database being null, means you never provided it in the first place, But for select they expect to find it , or find it null, empty. So instead of defining 2 type manually, or use any type, we used inferInsert an inferSelect, and all is done for us.

And then we insert :

```
await db.insert(schema.entities).values(EntityData);
```

Why this way; as I said earlier , my development idea, kept on changing, so I was deemed to come back in this file, and change seed, and as I use AI for randomised seeding, I need something that I can easily correct and fix , wrong entries myself.

And having a object, it s very convinient, I can easily se what am to change and change... without touch the insert function. And if the error , appears, it will not be an intengled error, it will b focusing only on this object here, which is not complying with the type I gave it.

5. 6.2 Seed your database

> so that is it, we now can migrate and seed things, push and seed ...

NB: but now I had too many scripts to run and these drizzle kit commands are ugly, easily forgatable. , I made a scripts to abstract them

```

"db:wipe": "tsx src/database/nuclearReset.ts",
"db:push": "npx drizzle-kit push",
"db:seed": "tsx src/database/seed.ts",
"db:studio": "npx drizzle-kit studio",
"seed:force": "npm run db:wipe && npm run db:push && npm run db:seed"

```

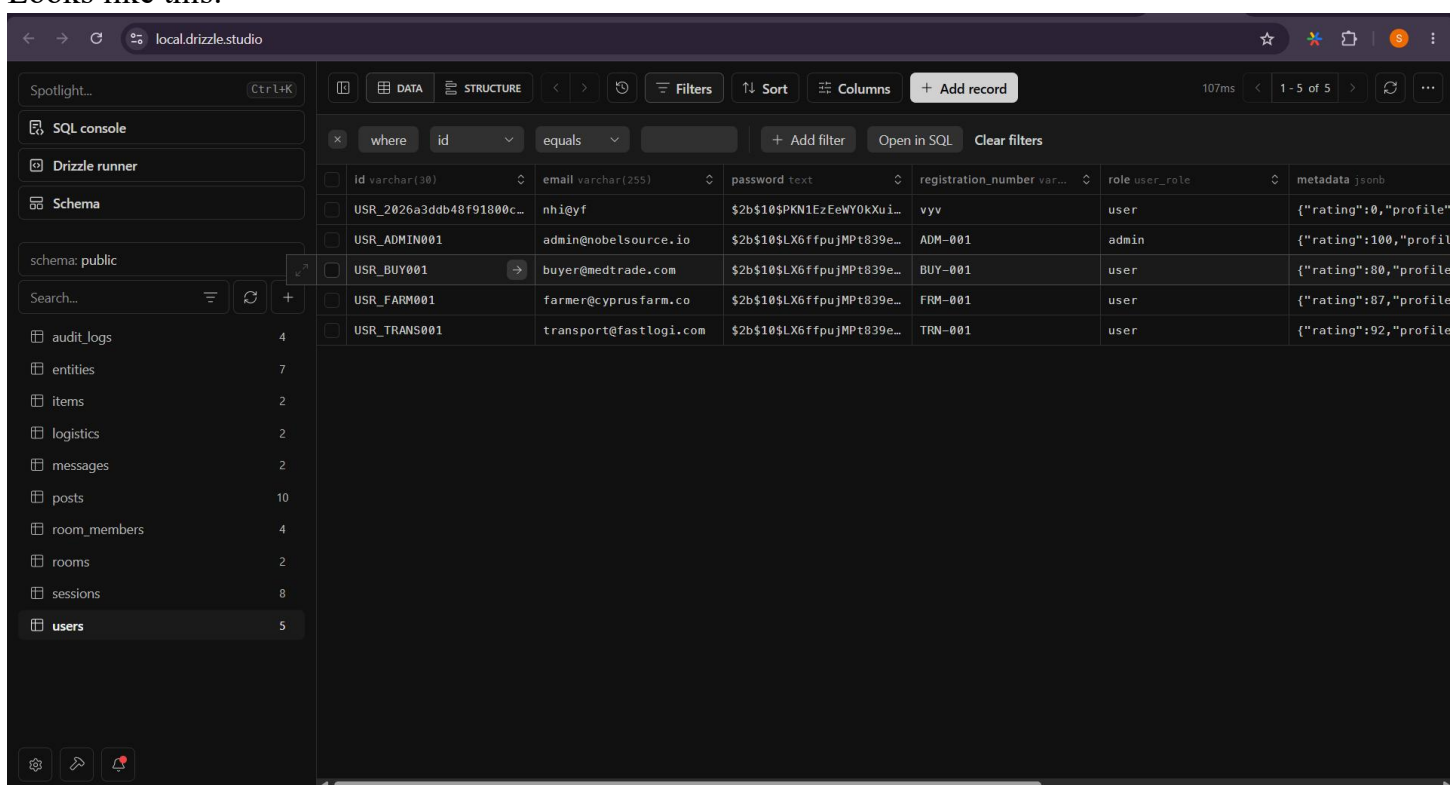
Wipe is is like the table cleaning but is also remove the enums, simply is a file, that on running it , I will

be like having a completely new database;
push ; you run this if you have any new thing in the schema, which unfortunately I was having more often that it should have been.

Seed : is the normal seeding, is running this file we created above , to populate our database with abit logic but fake data.

Studio: this is the number two, thing that attracted me, this is a GUI for my entire database, truly in rearly opened my terminal and launch my psql instance, s this studio , gave me the ui exactly the same as that of neon db; maybe as they are both for postgresql they brought that mentality to the local instance as well, which is very nice. But this studio is in beta , at least when I started my project, I never checked again, but it works fine.

Looks like this:



So to avoid all that headache I made a forcing script, where I combined everything in sequence once one fails the other one will clear the issue , like that when ever I was to seed I runn one command like script that always worked, no matter what, unless I have issue in my code. My be my intelisence is tired and it did not give red lines, and I missed it.. only those small things.

5.6.3 create your services

I mentioned about services earlier, the main intention is to talk to the database, as much as we want , in the service, get the response ready, so we can process them on the javascript side, and serve them through API endpoints.

My services class based classes

In general we need the drizzle-orm , to provide use with all those keywords tat mimic the sql behavior ; we need the db the connection instance and the table we are dealing with allong with its type.

```
import { eq, and } from "drizzle-orm";
import { db } from "../dataBase/db";
import { entities, type Entity } from "../dataBase/schema";
```

Then in the class we difine the table we have as we might need to use more than one table for a query.

Then we query in the drizzle way.

```
export class EntityService {
  private table = entities;

  async CreateEntity(data: Entity) {
    try {
      const [result] = await db
        .insert(this.table)
        .values({
          id: data.id,
          type: data.type
        })
        .returning();
```

```
      return { msg: "success", ans: result };
    } catch (e) {
      return { msg: e.message, ans: null };
    }
  }
}
```

As you can see, the best advantage of drizzle is that if one is familiar with sql, drizzle is only strengthening the discipline nothing much. And the strengthening part is not in your care, just do whatever you feel like it can work in sql, once you see red line, it will tell you what to change , as

the whole environment is well typed the error message will be clear enough , to know how to navigate, and happy coding...

NB: aside of the 10 table service, me I added the cloudinary service(my cloud bucket to keep my medias , for now in images and documents later on , maybe it can be something else. And auth service,

Cloudinary service

As we have pg for postgres , that help us to create the instance to our tabase, ; cloudinary has what we call **V2** , so we get it and me I kept calling cloudinary, as I don't even understand what V2 means, it is literally adding another confusion to my list.

And as we might upload more than one media content , we need to keep them in buffer of a certain stream , we get "streamfier" I beilive that is its use.

Then we configure the couldnary(cloud_name ; api_key and api_secret) the name is already visible when you inter you cloudflare, , the key and secret, you just firsrt create that api permission, in settings, then it will give you these two.

```
cloudinary.config({
  cloud_name: process.env.CLOUDINARY_NAME!,
  api_key: process.env.CLOUDINARY_KEY!,
  api_secret: process.env.CLOUDINARY_SECRET!
});
```

I have 3 services:

1. uploadBuffer : it take you buffer (groupe of files, and the name you want to call the folder that files will go in, just for cleanilness. Then we user the .uploader.upload_stream() function ; taking 3 parametarm , folder name , callback function that has error and result as parameter, to react when response is ready, and the resolve that output the information of a succes.

Then all this are attached to the streamfier.createReadStream(buffer).pipe(the who thing we prepaired).

```
static uploadBuffer(
  buffer: Buffer,
  folder: string
): Promise<{ url: string; publicId: string }> {
  return new Promise((resolve, reject) => {
    const uploadStream = cloudinary.uploader.upload_stream(
```

```

    { folder: `platform/${folder}` },
    (error, result) => {
      if (error || !result) return reject(error);

      resolve({
        url: result.secure_url,
        publicId: result.public_id
      });
    }
  );
};

```

```

    streamifier.createReadStream(buffer).pipe(uploadStream);
  });
}

```

2. Delete image : we only need the public URL Id , via the uploader, from cloudinary or from V2, we shall destroy it.

```

const result = await cloudinary.uploader.destroy(publicId);

```

3. Then the service of extracting public from URL, it is just string wise extract the last portion, which is the media name, and the folder it is found in.

Auth service

So we use many other service, to create the auth service, try to sequence things around so they make sense for proper secure authentication.

But as we are using other service why having it as a service not as a controller, where the orchestration of service happen? True here we are only orchestrating, but the level of orchestration if we push them to the controller level, there we will be dealing with users's request then things will get messy, we try to reduce the orchestration by the auth logic itself and to finish work after.

So here we made register, login , logout, soft delete and complete delete.

The complete delete we had to use a transaction, where, we had to remove the existence of this user in all places.

```

async fullyDeleteUser(targetUserId: string, adminId: string) {
  try {
    await db.transaction(async (tx) => {

```

```

    await tx.insert(audit_logs).values({
      id: `log_${Date.now()}`,
      entity_id: adminId,
      action: "ADMIN_DELETE_ENTITY",
      detail: { targetUserId },
      created_at: new Date()
    });

```

```

    await tx.delete(audit_logs)
      .where(eq(audit_logs.entity_id, targetUserId));

```

On the room

I created a fall back

```

    await tx.delete(rooms)
      .where(eq(rooms.id, targetUserId))
      .catch(() => {});

```

A side these services; I prepared a utility file ; with few utilities to use in any other files to come, like **setSessionCookie**

It keeps my token , under **session_token** key, and for a day, the othe parameter, I took them from documentation as they are nothing much.

```

export const setSessionCookie = (
  res: Response,
  token: string,
  options?: CookieOptions
) => {
  const isProd = process.env.NODE_ENV === "production";

  res.cookie("session_token", token, {
    httpOnly: true,

```

```

    secure: isProd,
    sameSite: isProd ? "none" : "lax",
    maxAge: options?.maxAge ?? 1000 * 60 * 60 * 24,
    path: "/"
  });
};

```

But cookies comes in the controller, not in service. Controller, deals with the http browse thing, but service should be working regardless, so we set session in services, by updating in the table. But cookies to be sent to the browser come in the controller.

And we have clear session cookie, where we clear cookie under then name **session_token**.

5.6.4 create your controllers

So here we are no free from database, all we focus on is to **reason with user's request** orchestrate the services, at large we have, and provide a **well structured response**.

```
import { AuthService } from '../services/auth';
```

```
const authService = new AuthService();
```

```
import { clearSessionCookie, hashPassword, setSessionCookie } from
'../services/utils';
```

```
const hashedPassword = await hashPassword(raw.password);
```

```

static async login(req: Request, res: Response) {
  const { email, password } = req.body;

  const result = await authService.loginUser(email, password);

```

As you can see the work keep on reducing and the separation of concern is clear, so why when I was changing things I was safe, whatever that can break it will break when I hit it, otherwise things are well isolated, it will not be a spaghetti code, where solving one thing breaks the other, no one can make sense of what is happening.

Middle wares:

But we have the session and the main reason of session is to remember who we are, so we need to define another kind of request , that upon login in , or session is created, the user information, backend will remember it.

```
export interface AuthRequest extends Request {
  userContext?: {
    token: Session["token"];
    entity_id: Session["entity_id"];
    expires_at: Session["expires_at"];
  };
}
```

So this is done in the middleware, this Is the whole mechanism of checking if we are logged in or not.

```
export const authGuard = async (
  req: AuthRequest,
  res: Response,
  next: NextFunction
) => {

  const token = req.cookies?.session_token
  if (!token) {
    return res.status(401).json({ msg: "No token provided" });
  }

  const result = await sessionSerive.validateSession(token);
  if (!result.ans) {
    return res.status(401).json({ msg: result.msg || "login first" });
  }
  req.userContext = {
    token: result.ans.token,
    entity_id: result.ans.entity_id,
    expires_at: result.ans.expires_at
  }
}
```

```
};  
  next();  
};
```

Multer & allowed formats

```
import multer from "multer";  
  
const storage = multer.memoryStorage();
```

```
const allowed = new Set([  
  "image/jpeg",  
  "image/png",  
  "image/webp",  
  "application/pdf"  
]);
```

```
export const upload = multer({  
  storage,  
  limits: {  
    fileSize: 10 * 1024 * 1024 // 10MB  
  },  
  fileFilter: (req, file, cb) => {  
    if (allowed.has(file.mimetype)) {  
      cb(null, true);  
    } else {  
      cb(null, false);  
    }  
  }  
});
```

Not much I understand here but, what I can tell is that , we have 10md per file and we support, images(jpeg,png,webp) and pdf files.

5.6.5 create your route

Here the intention is to define how are we receiving the user request, in what format, who is allowed to access this endpoint, what are the prerequisites, . simply this is the guard of the api world.

We will assign the endpoint to the right controller to serve it.

And if we need any middleware to run in many endpoint we will **use** it and put all to be affected endpoint beneath.

```
const router = Router();

router.post(
  "/register",
  upload.fields([
    { name: "display_image", maxCount: 1 },
    { name: "permissions", maxCount: 20 }
  ]),
  AuthController.register
);
```

In the above we used upload middleware, to receive 2 entries under permissions name, and one entry under name display_image. **Once we use multer automatically we use form_data.**

Then we connected this endpoint to the controller AuthController, on its static function register.

```
router.post("/login", AuthController.login);
```

This is the normal way if no multer is in place, we use the instance of Router() object, and under our method POST we link it to the path “/login” which adds up to anything our parenting route contains, then we give the function to process the request, which is in the controller.

```
router.use(authGuard);

router.post("/logout", AuthController.logout);
```

Now anything underneath authGuard, they must satisfy whatever authGuard is requesting. Which is to have the token in the AuthRequest, as we saw up, in the auth middleware. If not it can create it for you, if the token is still valid, or if you don't even have token created for you, means you never logged in, you need to do so.

Why the /api ?

Believe me I always asked my self why people like to complicate like and add a /api at their end point, and I saw it here. Not all end points are designed to server as medium for data in and out,

As we have many routers, we must loose track of what is available and what is not, and maybe later on , we might like to run the same architecture but ship a different product,

means when we run this server instead of serving this one backend we can serve another , gaming project maybe, or any other, we group things in one central router in,

```
const router = Router();

router.get('/', (req, res) => {
  return res.json({
    status: 'ok',
    message: 'Welcome to the Nobelsource API',
  });
})
```

```
// semi - public
router.use('/auth', authRoutes);
router.use('/posts', postRoutes);
router.use('/rooms', roomRoutes);
```

```
// protected
router.use(authGuard);
```

```
router.use('/users', userRoutes);
router.use('/audit', auditRoutes);
router.use('/logistics', logisticRoutes);
```

```
router.use('/entities', entityRoutes);
router.use('/messages', messageRoutes);
```

```
router.use('/rooms', roomMemberRoutes);
router.use('/items', itemRoutes);
```

Here we defined on the high level, what is guarded what is not., and this file that is groping things together , it is the one we shal link to the **api** route. So api route is for cleaniliness where , we ned to properly organinse our interface.

5.6.5 Serve apis, & api documentation

So we just server out api, which reside in out index.ts which is this above comnined route. And define api documentaion.

Fisrt we need to bring in Express; set up cors (my two , ther four times bug , I learnt it the hard way) , and as we have cookies in the picture , we need cookie parser .

steps

1. Instance the express

```
const app = express();
```

2. Set cors

```
app.use(  
  cors({  
    origin: [  
      "http://localhost:3000",  
      "http://localhost:5173",  
      "http://localhost:3001",  
    ],  
    credentials: true,  
  })  
);
```

3000 is for a fellow backend using express , to communicate to us, or if it is miss configured we have 3001, then the main, client of ours, 5173 it is a vite port.

3. Use parsers to read anything incoming.

```
app.use(express.json({ limit: "10mb" }));  
app.use(express.urlencoded({ extended: true, limit: "10mb" }));  
app.use(cookieParser());
```

4. Server the public folder as static, so that I can render my api documentation

```
const __filename = fileURLToPath(import.meta.url);
```

```
const __dirname = path.dirname(__filename);

app.use(express.static(path.join(__dirname, "..", "public")));
```

5. Serve the router

```
app.use("/api", router);
```

6. Listen on the port (3000)

```
app.listen(PORT, () => {
  console.log(`🚀 Server running on http://localhost:${PORT}`);
});

export default app;
```

5.7 Frontend Architecture

Frontend was quite smother, than the backend , as the configuration was a pre-made thing. I go it from schaden, I mentioned it above.

For the present of the project it comes with:

- Configured tailwindcss
- Context provider for them toggle
- Color schema for my ui already made in the css. Both for the light and dark theme.

So me I started by setting my routes., use tan stack query (the best thing am proud of so far, this was a life saver, caching out of my concern, fetching is clean, no more over head)

Set up for the main .tsx

- import the app.tsx, the one responsible of rendering the pages
- import all pages, so we can connect them with the path.

```
import { MarketFeed } from "../pages/MarketFeed.tsx"
import { Login } from "../pages/login.tsx"
```

Connect them to paths

```
const router = createBrowserRouter([
  {
    element: <App />,
    children: [
      {
        element: <AuthLayout />,
        children: [
          { path: "/", element: <SplashScreen /> },

```

```

    { path: "/login", element: <Login /> },
    { path: "/register", element: <Register /> },
    { path: "/Chamber/:roomName", element: <RoomInterior /> },
    { path: "/test", element: <Test /> },
  ],
},

```

Here I used layout approach where, some pages were to have navbar and footer, but other would not. This matters much for me as in this system I intend to present 2 different feeling, the public feed, where it is all about posting, and connecting with fellow seller, buyers, but when you enter your room, it should be more professional, more serious, so we should not see the footer, the navbar. We should leave the room totally free from irrelevant contents. That is why of these layouts.

So here we pass the routes here

```

<RouterProvider router={router} />

```

➤ Import tanstack/react-query, this will help us to fetch data.

```

import { QueryClient, QueryClientProvider } from "@tanstack/react-query"
const queryClient = new QueryClient()

```

```

<QueryClientProvider client={queryClient}>
  <ThemeProvider defaultTheme="system" storageKey="app-theme">
    <RouterProvider router={router} />
  </ThemeProvider>
</QueryClientProvider>

```

So that is how. Also the theme provider is the global context for theme toggle.

Set up for the App.tsx and Layouts

- Here nothing much happened we prepare the Outlet so that the layout will Rerender according to the routes.
- I also created my own global context the one, brought along with the present it was, in change of the theme toggle, and a lot of other thing, I was afraid of dealing what may destroy a lot. So I know how to create my own global context, so I created mine.

```

import { Outlet } from "react-router-dom"
import { GlobalProvider } from "../utils/Hooks/General"

function App() {
  return (
    <GlobalProvider>
      <Outlet />
    </GlobalProvider>
  )
}

```

```
export default App
```

These are one of the layout:

```
import { Navbar } from "@components/Navbar"
import { Outlet } from "react-router-dom"

export const MainLayout = () => {
  return (
    <div className="flex min-h-screen flex-col items-center justify-center">
      <Navbar />
      <Outlet />
    </div>
  )
}
```

Prepaire the query option.

Tanstack operates on “options” which is the key which caches you queried data by default, and i don’t know for mutation but the documentation says that for mutation we don’t cache so , I accepted that way.

But now to keep having to write the fetch function multiple times it was tedious so I created and generic fetch function , that will accept all methods, and any find of data.

Options/FetchTemplate.ts

```
import type { BodyProps } from "@types/api"
import type { Methods } from "@types/general"

export const FetchTemplate = async <T>(endpoint:string,method:Methods = "GET",body?: BodyProps):Promise<T> =>{
  const isFormaData = body instanceof FormData;
  const Headers: HeadersInit = {};
```

```
  if (body && !isFormaData) Headers['content-type'] = 'application/json';
```

```
  try{
    const response = await fetch(
      endpoint,
      {
        method,
        credentials: "include",
        body:isFormaData?body:JSON.stringify(body),
        headers:Headers
      }
    );
```

```
    if (!response.ok) {
      const err = await response.json()
```

```

        console.error("BACKEND ERROR:", err)
        throw new Error(err.msg || "Request failed")
    }else{
        return response.json()
    }
}catch(err){
    return {error:err} as T
}
}

```

Explanation:

we take in parameter: endpoint url , methods defaulted to GET, as for this prototype, I planned to mostly get most things, , body I clude have said any but this is delicate , it is the fore front line to talk to the backend , if we are loose here, that is the two side not reaching to each other, and anything broken it may break the backend, which took me too long. So I use type enumeration.

```
type BodyProps = Login | Register | LoginResponse | RegisterResponse
```

So we have in the backend to submit a formdata, when it involve media, or a normal json , So I had to check if we receive a formData if not it is defenetly a json.

So we just fetch , so nothing much else.

Prepaire your option and the function.

1. QueryOption

```

export const GetRoomLogisticsFN = async (roomId: string):
Promise<RoomItemDetail[]> => {
    return FetchTemplate<RoomItemDetail[]>(
        `${BaseApiUrl}/logistics/${roomId}`,
        "POST"
    );
};

export const GetRoomLogisticsOption = (roomId: string) => {
    return queryOptions({
        queryKey: ["rooms", "logistics", roomId] as const,
        queryFn: () => GetRoomLogisticsFN(roomId),
        staleTime: 1000 * 60 * 5,
    });
};

```

So whatever we fetch here under GetRoomLogisticsOption the data we get there, it will be automatically cached ; so from the option we connect it with the function to run the specific fetch. We also may add on the staleTime, which is how long this cached data is valid for , so we do a refetch. Truly I don't even know how to do this manually , I do understand that caching fover , I does not make sence as backend keep cahnging , so refetch is a must, but how to do it manually, I would have spent weeks maybe trying to know how, but tansstack, show me the best way to learnt and appreciate the tools out there.

2. MutationOption

```
const LogoutFN = async ():Promise<Logout> =>{
  await new Promise((resolve) => setTimeout(resolve, 500))
  return await FetchTemplate<Logout>(
    BaseApiUrl+"/auth/logout",
    "POST"
  );
}
export const LogoutOption = () =>{
  return mutationOptions({
    mutationKey: ["Logout"] as const,
    mutationFn: LogoutFN,
  })
}
```

Mutation as you can see nothing much change for what the programmer does, only that we are not chaching these logout,

Why making the timeout, as the server is running on this pc, exactly, it is very near there is not much of traffic, so the post fetch will go extremely fast it is not like we are getting a huge list of things. So it will not load, I had to mimic load behaviour so I can provide the ui for that case, if I did not I would build a product good for local but worst for deploy.

Use the options in the Pages and components

```
const { mutate, error, isPending } = useMutation({
  ...LoginOption(),
  onSuccess: (data) => {
    if (data.msg === "success") {
      localStorage.setItem("user", JSON.stringify(data.ans.user))
      navigate("/MarketFeed")
    }
  },
  onError: (err) => {
    console.error("Login failed:", err)
  },
})
```

And query

```
const {
  data: messages = [],
  isLoading,
  isError,
} = useQuery(GetMyMessagesOption())
```

NB: but the way we trigger these, options or fetches differs, there is what you can do and what you can not do.

There is a state based trigger : for exapnad if test is true we will trigger the fetch:

```
const [test, setTest] = useState(true)
const {
  data: messages = [],
  isLoading,
  isError,
} = useQuery({ ...GetMyMessagesOption(), enabled: test })
```

Not only this we can also start together the fetches, once one is done we trigger the other one.

```
const { data: me, isError: me_Error } = useQuery({
  ...MeOption(),
})

const {
  data: messages = [],
  isLoading,
  isError,
} = useQuery({ ...GetMyMessagesOption(), enabled: me_Error === false &&
me?.user !== undefined })
```

But the mutation can not allow these, we can not even start things together, we only have to call mutate() when we need to trigger it. And pass in data we need to send.

```
const handleLogin = async (e: React.FormEvent<HTMLFormElement>) => {
  e.preventDefault()

  mutate({
    email,
    password,
  })
}
```

We can put it in the useEffect, any where you can call mutation, you can. Maybe there is a way around to make a chain fetches in mutation but it will be tricky, as the architecture is not built to support it, but anything in code, always has a way, “may be hard or easy, but code are transparent.

NB: I used from motion but not the usual way of defining every variant every where; I defined them in one area, and re-used; whenever in need, making the website having that same feeling:

```
export const heartBeat: Variants = {
  animate: {
    scale: [0.8, 1, 1.05, 1.1, 1.05, 1, 0.8],
    transition: {
      duration: 1.5,
      repeat: Infinity,
      repeatType: "reverse",
      ease: "easeInOut",
    }
  }
}
```

```

}
export const fadeIn: Variants = {
  initial: { opacity: 0 },
  animate: { opacity: 1},
  exit: { opacity: 0}
};

```

The prepare the motion wrappers.

```

export const Pulse = ({ children, className }: Props) => (
  <motion.div variants={heartBeat} animate="animate" className={className}>
    {children}
  </motion.div>
)

```

And use it

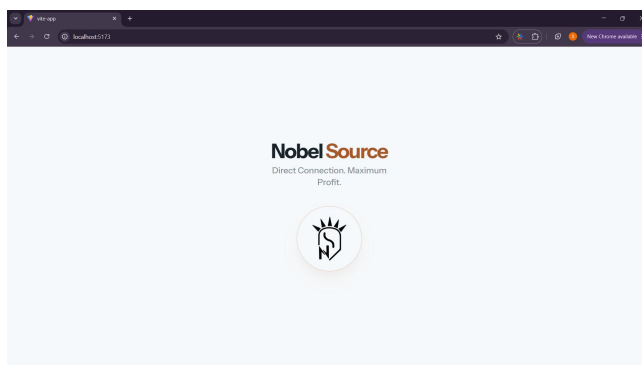
```

<Pulse className="relative z-10 flex h-48 w-48 items-center justify-center rounded-full border-2 border-primary/20 bg-background shadow-2xl shadow-primary/10">
  <img src={logo} alt="Logo" className="h-32 w-32 object-contain" />
</Pulse>

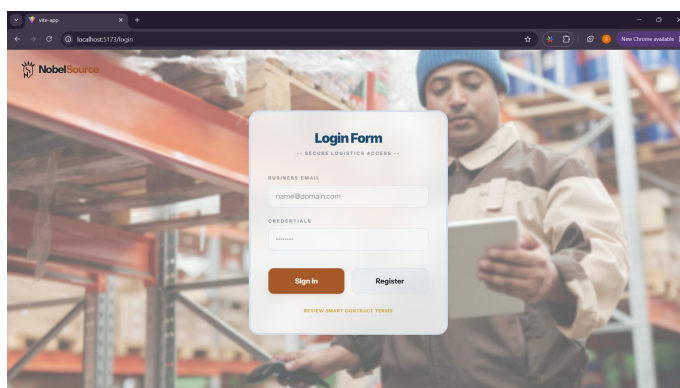
```

6. Website Overview

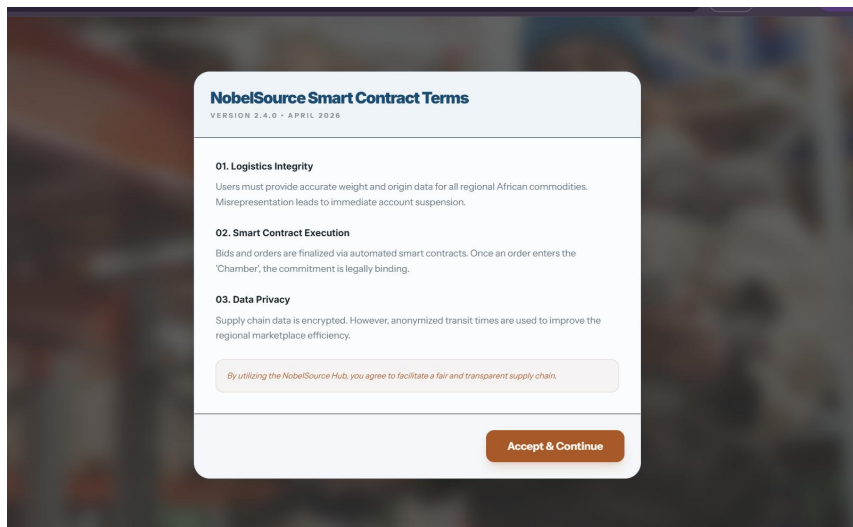
Splash screen



Login



Clicking on nobel source terms



Registration

1.

The image shows a mobile application screen for registration. At the top, there is a logo for "NOBEL SOURCE" and a "Cancel" button. Below the logo, there is a progress bar. The main section is titled "Credentials" and contains three input fields: "EMAIL" (with the value "try@gmail" and a character count of "9 / 100"), "PASSWORD" (with a character count of "3 / 100"), and "CONFIRM PASSWORD" (with a character count of "3 / 100"). To the right of these fields is an "APPLICATION AUDIT" section. This section has a table with three rows: "Account" (try@gmail, status: success), "Identity" (Missing, status: error), and "Verification" (Missing, status: error). Below this table is a section titled "SELECTED INDUSTRIES" with the text "No tags added". At the bottom, there are "Back" and "Continue" buttons.

NOBEL SOURCE Cancel

Credentials

EMAIL
try@gmail
9 / 100

PASSWORD
...
3 / 100

CONFIRM PASSWORD
...
3 / 100

APPLICATION AUDIT

Account	try@gmail	✓
Identity	Missing	!
Verification	Missing	!

SELECTED INDUSTRIES
No tags added

Back Continue


Cancel

Profile Setup

BUSINESS NAME

BrunoLtd

8 / 100

PERSONAL SUMMARY

am a student of software engineering, helloam a student of software engineering, helloam a student of software engineering, helloam a student of software engineering, hello

215 / 500

PHONE (OPTIONAL)

0790000149

10 / 100

COUNTRY (OPTIONAL)

Cyprus

6 / 100

WEBSITE (OPTIONAL)

www://localhost:-2/testme.com

29 / 100

BUSINESS AVATAR

Choose File pexels-olly-3777557.jpg

pexels-olly-3777557.jpg

Back
Continue

APPLICATION AUDIT

Account

try@gmail

✓

Identity

BrunoLtd

✓

Verification

Missing

!

SELECTED INDUSTRIES

No tags added



New Entity

2.


Cancel

Seal of Integrity

JOB TITLE

CEO

3 / 100

REGISTRATION NUMBER

REG_12342

9 / 100

ADD INDUSTRY EXPERTISE

CERTIFICATION NAME

0 / 100

UPLOADED DOCUMENTS

scammer

Choose File

GBYF-KIBRIS_Ör...e6_page-0001.jpg

GBYF-KIBRIS_Örnek Proje P...

2280.3 KB

VIEW

APPLICATION AUDIT

Account

try@gmail

✓

Identity

CEO_BrunoLtd

✓

Verification

REG_12342

✓

SELECTED INDUSTRIES

scammer

✓



BrunoLtd

Back
Continue

3.


Cancel

Final Review

Title: CEO
Name: BrunoLtd

summary:
am a student of software engineering, helloam a student of software engineering, helloam a student of software engineering, helloam a student of software engineering, helloam a student of software engineering, hello

Review the sidebar audit to ensure all green checks are present.

Back
Submit Application

APPLICATION AUDIT

Account try@gmail	✓
Identity CEO_BrunoLtd	✓
Verification REG.12342	✓
SELECTED INDUSTRIES	
scammer	✓



BrunoLtd

4.

Then submit, as it need to talk to cloudnary it takes a while, even we are on the same computer:


Cancel

Final Review

Title: CEO
Name: BrunoLtd

summary:
am a student of software engineering, helloam a student of software engineering, helloam a student of software engineering, helloam a student of software engineering, helloam a student of software engineering, hello

Review the sidebar audit to ensure all green checks are present.

Back
Submitting...

APPLICATION AUDIT

Account try@gmail	✓
Identity CEO_BrunoLtd	✓
Verification REG.12342	✓
SELECTED INDUSTRIES	
scammer	✓



BrunoLtd

You are sent in the market feed:

Chamber

Notifications

Profile

MARKET FEED

USD

Search assets...

ALL SECTORS

OFFER

PRODUCER_ALI DE... +1

PREMIUM CYPRUS POTATOES - 20 TONS

Red-skin potatoes, washed and export-ready from Güzeluyurt.

GOZELYURT FARM HUB, TRNC

20 /kg USD

WANT

PRODUCER_ALI DE... +1

BULK CEMENT SUPPLY NEEDED

Looking for 500 bags of Portland Cement for Nicosia site.

NICOSIA CONSTRUCTION ZONE

15 /Item USD

OFFER

PRODUCER_ALI DE... +1

COLD STORAGE LOGISTICS SERVICE

Available reefer trucks for chilled transport to Limassol Port.

FAMAGUSTA LOGISTICS PORT

20 /hour USD

OFFER

PRODUCER_ALI DE... +1

ORGANIC OLIVE OIL - 500L

First cold press extra virgin olive oil from Kyrenia mountains.

KYRENIA OLIVE PRESS

30 /liter USD

WANT

PRODUCER_ALI DE... +1

NEED SOLAR PANEL INSTALLERS

Seeking certified team for 10kW residential installation.

KYRENIA HEIGHTS

150 /hour USD

OFFER

PRODUCER_ALI DE... +1

EXPORT QUALITY LEMONS - 5 TONS

Freshly picked Meyer lemons from Morphou orchards.

MORPHOU PACKING HOUSE

18 /kg USD

Click on the deal to see more

ORGANIC OLIVE OIL - 500L

"First cold press extra virgin olive oil from Kyrenia mountains."

MARKET SPECIFICATIONS

Acidity below 0.8%. Harvested Nov 2025. Packaged in 5L tin containers. Laboratory testing results available upon request. International shipping available.

KYRENIA OLIVE PRESS

VALUATION

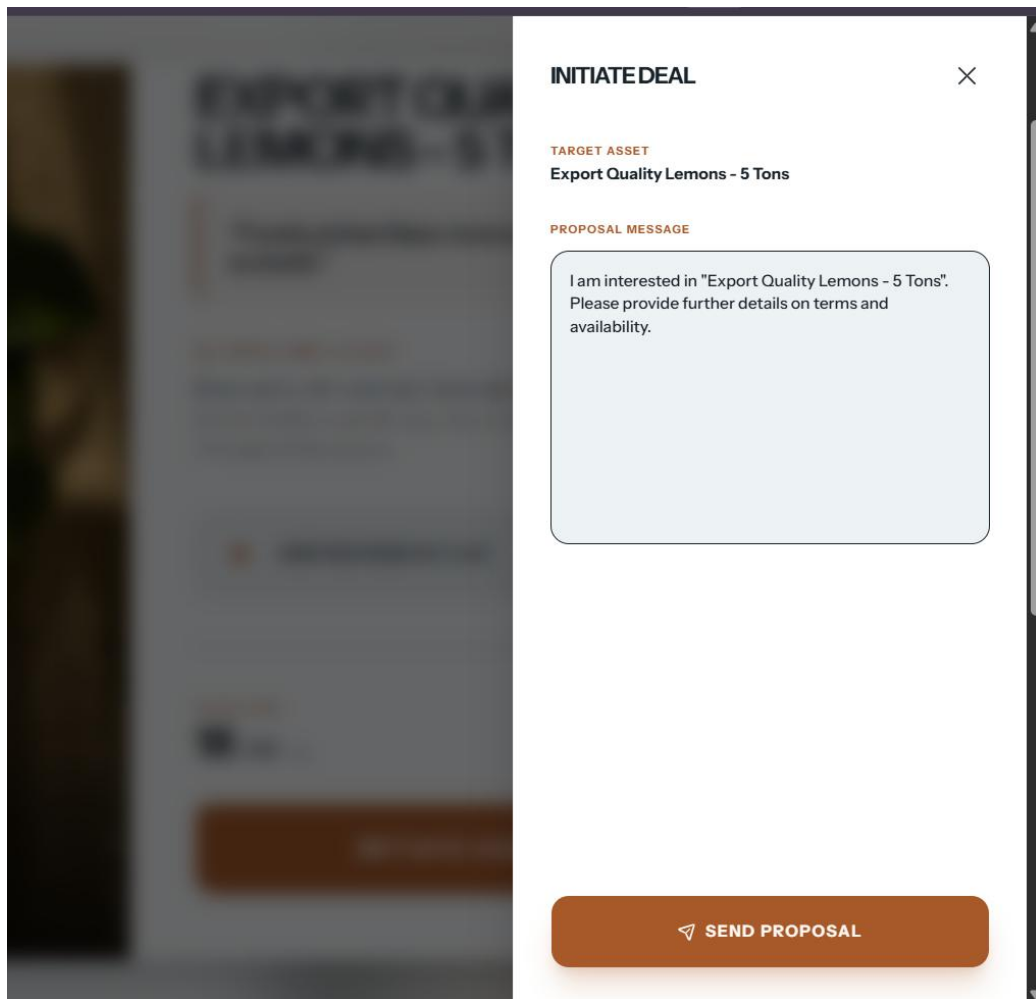
30 USD /liter

INITIATE DEAL

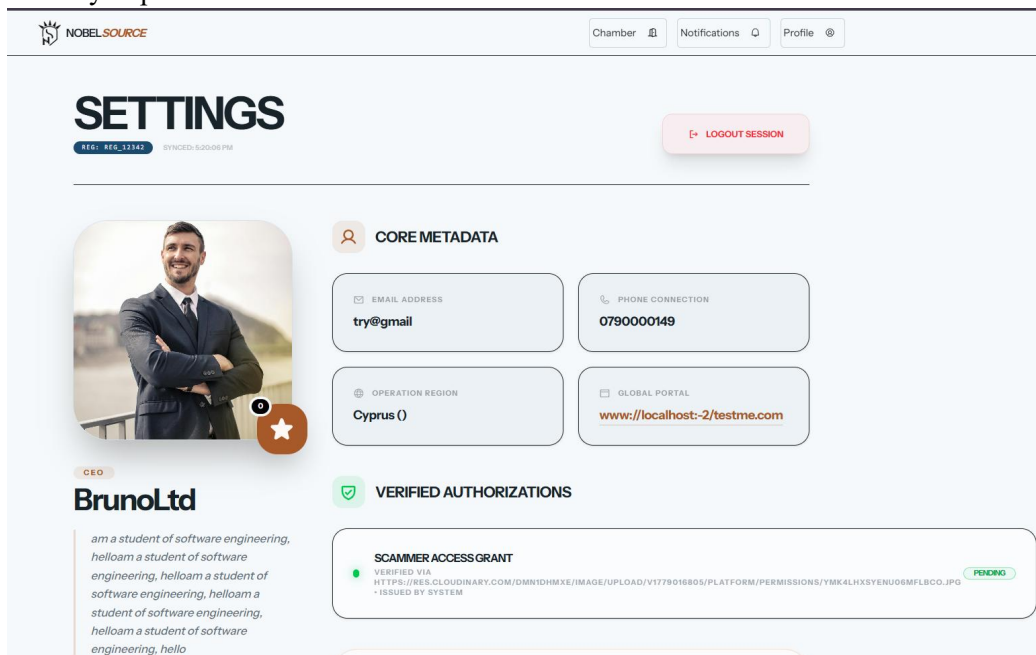
OFFER

REF.POST_004

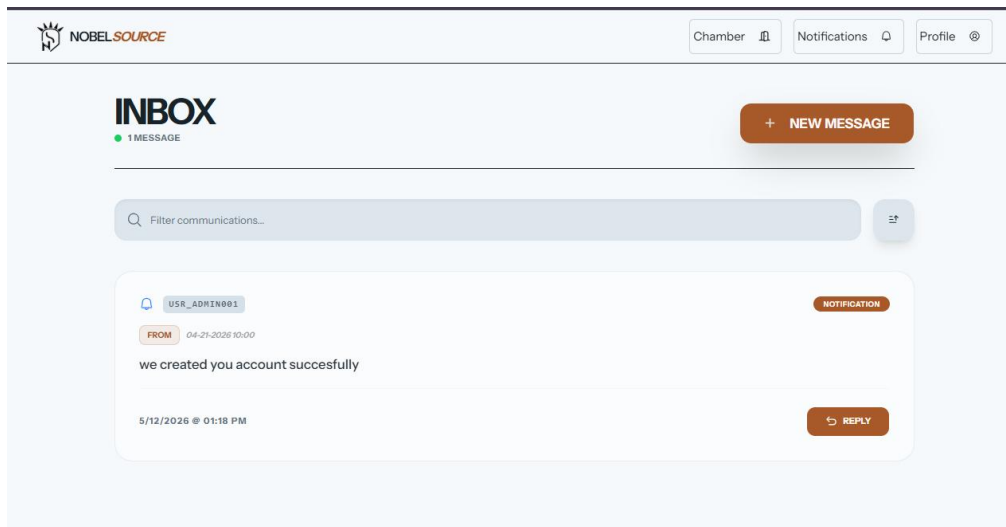
Send message if interested



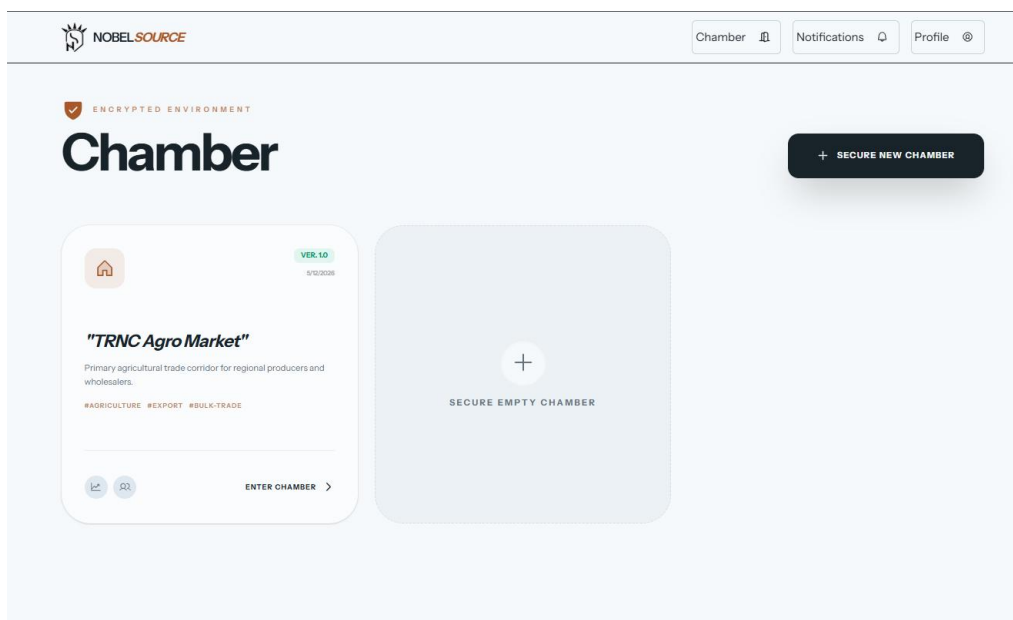
Check you profile



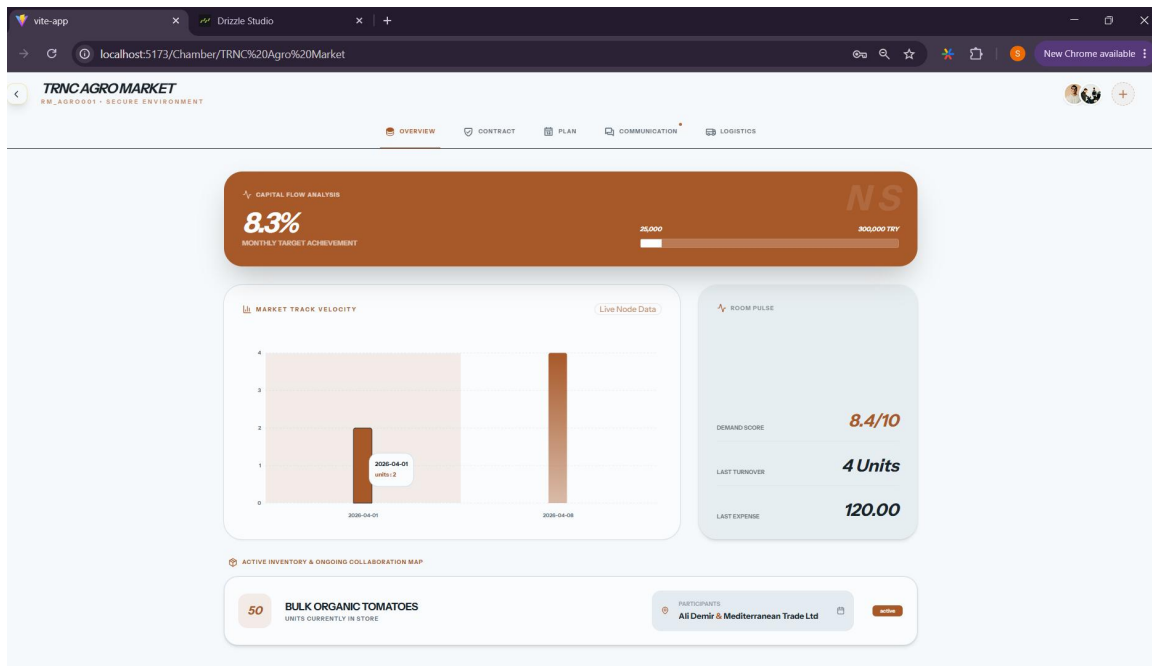
Check the global normal messages



Now going in real business, time to change the mood, less playfull design abit serious and defined



2 shipments , and we are at 8.3% on the cashflow we planned to have in this month



Our working contract, we signed , we can raise any issue for disute any time.

The screenshot shows the TRNC AGRO MARKET contract page. The header includes the company name and a navigation bar with links for OVERVIEW, CONTRACT, PLAN, COMMUNICATION, and LOGISTICS. The main content area is titled "LEGALLY BINDING PROTOCOL V1.0" and contains the following text:

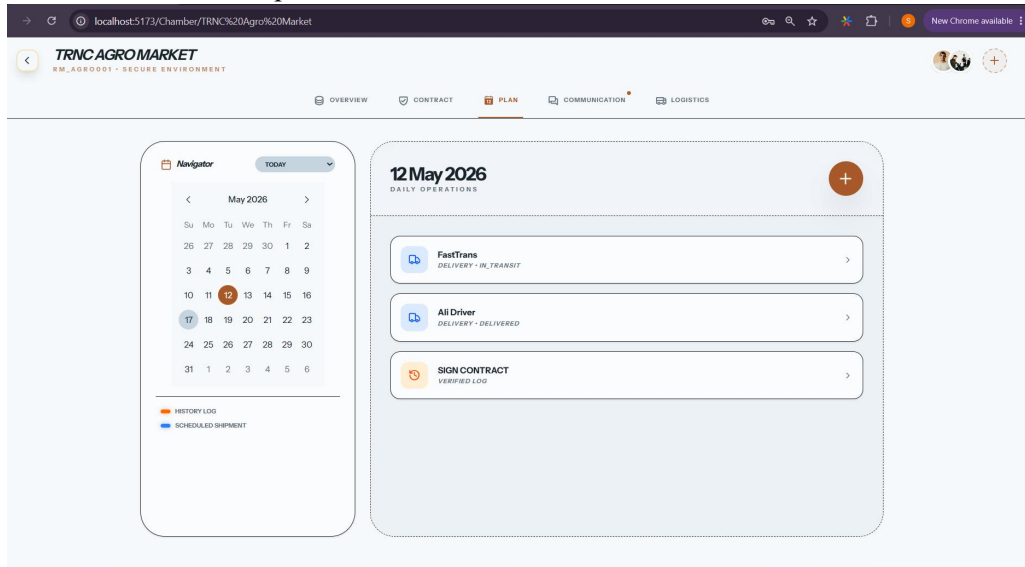
"This protocol governs the direct wholesale exchange of perishable agricultural goods between local farms and certified distributors."

The page is divided into four sections, each with a title and a description:

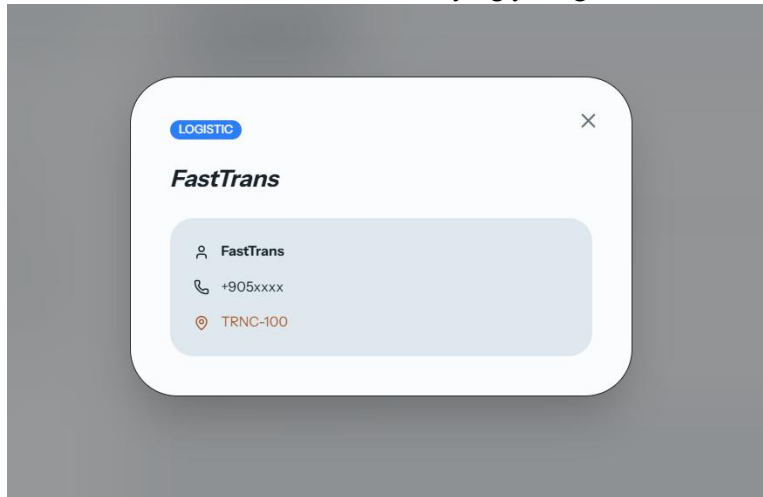
- CLAUSE 1: PENALTIES**
 - Default & Penalties
 - Late Delivery: 0.5% reduction in total value per day delayed.
 - Cancellation: If cancelled after truck dispatch, buyer forfeits 20% of total value as a logistics restocking fee.
- CLAUSE 2: PAYMENT TERMS**
 - Settlement Protocol
 - Escrow: Funds are locked upon order confirmation.
 - Release: 100% payout triggered by "Confirmation of Receipt" in the Logistics Tab.
 - Currency: All trades fixed in TRY to avoid exchange volatility.
- CLAUSE 3: SHIPMENT RULES**
 - Logistics & Handling
 - Incoterms: EXW (Ex Works) unless otherwise stated in the Item Analytics.
 - Quality Control: Goods must be inspected within 6 hours of arrival. Late claims will not be processed.
 - Packaging: Standard export-grade crates only.
- CLAUSE 4: DISPUTE RESOLUTION**
 - Arbitration
 - All disputes to be mediated by the TRNC Chamber of Commerce via the platform admin interface.

At the bottom, there are two buttons: "RAISE PROTOCOL ISSUE" and "PROTOCOL SIGNED".

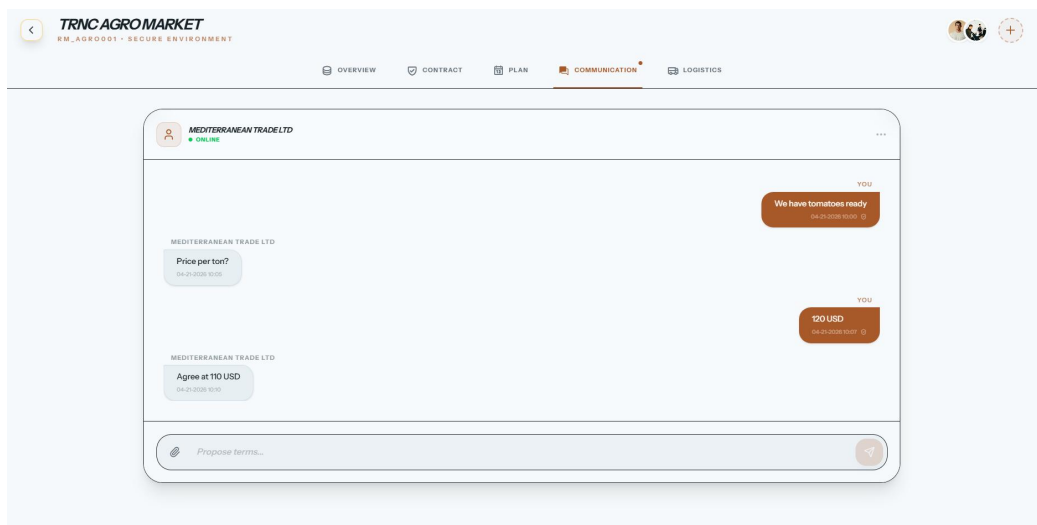
On 12th there was a shipment we received, and one still in transit, but we are on 17th



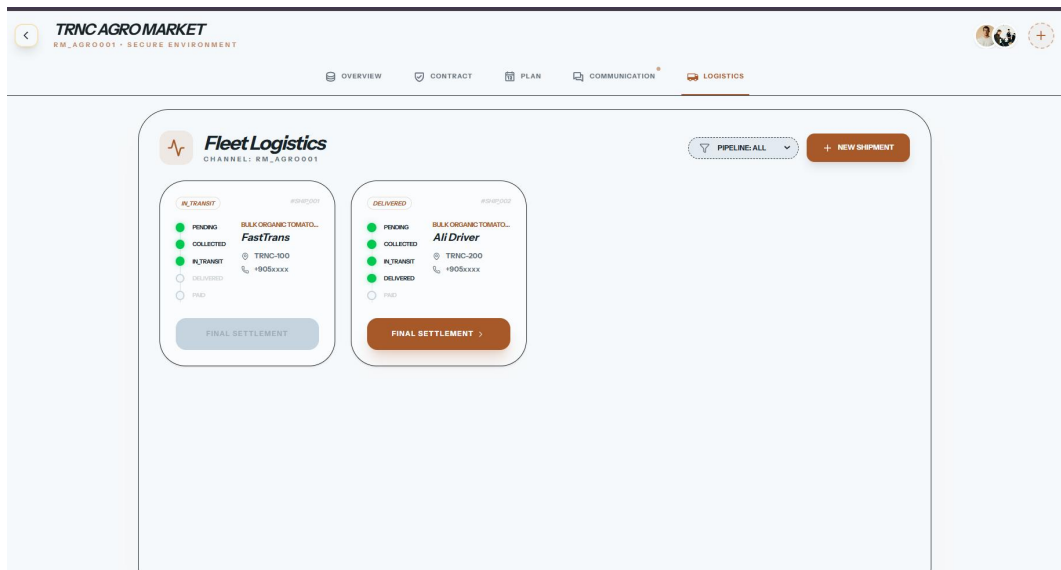
This is the information of the one carrying your good, we can ask,



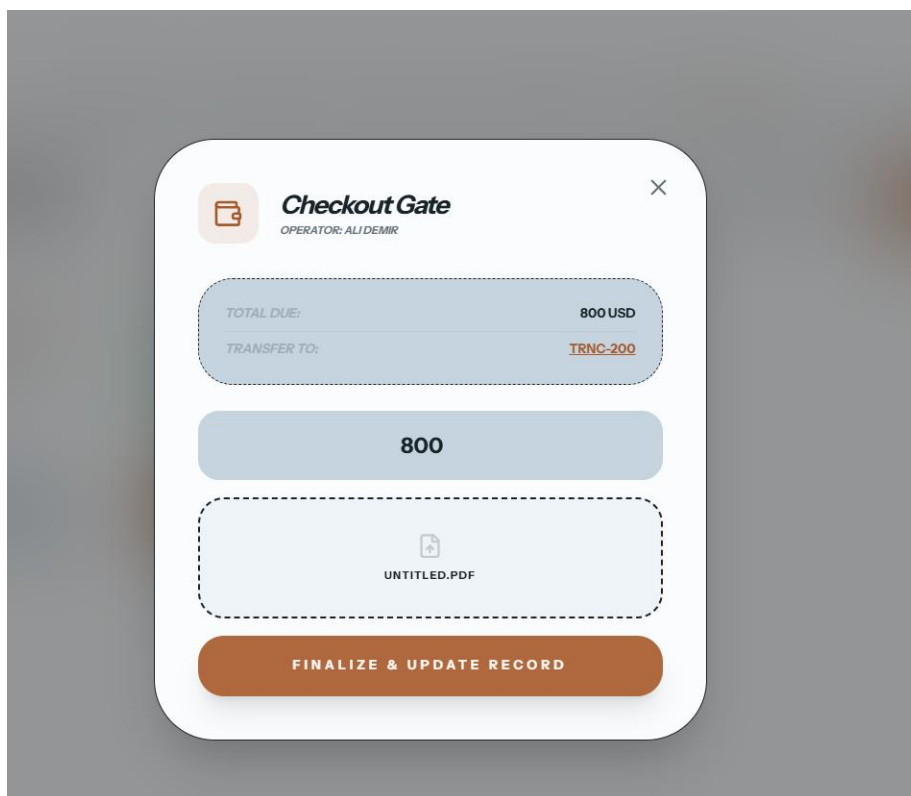
Communication



Good we are due to pay for , and those nearerly to arrive



Final settlement



With a proof on “untitled.pdf”

Then it will update the overview and that we completed another shipment, the cash flow will adjust and your trust score adjusted. And be able to keep working. With others.

Admin panel, I will build it soon, or maybe be in later development, it is not rushy as I have drizzle studio, which works for me well. It will depend.

7. Conclusion

7.1 Benefits

a. Benefits to users :

1. Predictable Income for farmers

Farmers who use Nobel Source know the price and volume before planting. If the buyer agrees to purchase 100kg per week at 50 KES per Kg, the farmer knows expected weekly revenue of 5,000 KES. This predictability enables investment decisions (buying better seeds, hiring labor) that were previously too risky.

2. Verified supply

Buyers who use Nobel Source know that suppliers have committed volume and delivery schedules. The trust score system shows which suppliers have completed deals on time. A buyer can filter for suppliers with >90% on-time delivery and avoid those with dispute histories.

3. Dispute Evidence that saves Time

When a dispute occurs, both parties upload evidence to the same system. The farmer's "shipment started" timestamp and the buyer's "shipment closed" timestamp (or absence) are visible to both. This eliminates the "you never sent it" / "you never confirmed" argument. The pre-agreed arbitrator (cooperative leader, chamber of commerce) reviews evidence and decides. The process takes days instead of months.

b. Benefits to me (the developer):

1. Event-Sourcing Without Event Store complexity

Implementing event interpretation on top of standard postgresql table taught me trade-off analysis: when to add schema columns (for high-frequency queries) versus when to encode meaning in JSONB fields (for flexible, low-frequency access). This skill is directly application to real-world system design where requirement change faster than databases can migrate.

2. Full-stack Typescript integration

Maintaining consistent Typescript types across backend (Node.js, Drizzle ORM), interpreter layer, and frontend (React) required disciplined type definition. Shared type packages (@nobelsource/types) ensure that changing a field in the database schema triggers compile errors in all dependent components until updated. This catches bugs that would otherwise reach production.

3. Realistic Scope Management

The most valuable learning was what to leave out. The project explicitly does not handle payments, does not provide legal enforcement, does not promise to eliminate market crashes. Defining these boundaries forced clarity about what the system actually does. Many student projects fail because they try to do too much. Nobel Source succeeds within its narrow scope.

7.2 Ethics

1. Honest claims about what the system does not do

The platform states clearly: "we do not fix everything. No platform can. But we fix the information gap." This honesty is encoded in the user agreement, displayed on the landing page, and repeated in help documentation.

Users cannot reasonably believe that Nobel Source provides legal enforcement, payment processing, or logistics guarantees because these claims are explicitly disclaimed [24].

Why this matters: Overpromising is common in agritech. Startups claim to “revolutionize agriculture” while delivering basic listing platforms. When farmers rely on these promises and lose money, trust in digital solutions erodes for everyone. Nobel Source avoids this by under-promising and over-delivering on its narrow scope.

2. Neutral Dispute Logging, Not Judging

When dispute occur, the system logs evidence but does not decide outcomes. A pre-agreed arbitrator(cooperative leader, chamber of commerce, trusted third party) makes the decision. This separation of concerns prevents platform bias. The same system serves both farmers and buyers- if the platform decided disputes, it would have incentive to favor whichever side pays more[25]

Implementation: The arbitrator is selected during contract creation. Both parties must approve. The decision is recorded in audit_logs with flag='critical'. Neither party can change the arbitrator after a dispute is filed.

3. Data Ownership and no monetization of user data

The platform does not sell user data, does not show third-party ads, and does not use trade data for purpose other than platform operation. Farmer's production volumes, buyers' purchasing patterns, and pricing information belong to the users who generated them[26].

Why this matters: Agricultural data is commercially sensitive. A buyers' purchasing patterns reveal business strategy. A farmer's production volumes reveal capacity. Selling this data would violate the trust that the platform depends on. Nobel Source operates on a subscription or transaction fee model (if commercialized) , not on data monetization.

4. Accessibility for Low-Tech Users

The platform is designed for basic smartphones with intermittent internet. All critical functions (marking shipment start, confirming receipt, uploading payment proof) work offline and sync when connectivity returns. No feature requires high-bandwidth video streaming or real-time GPS tracking[27].

Why this matters: Farmers who need the platform most are those with limited connectivity. Designing for high-bandwidth urban users excludes the rural producers who are the “ source” in Nobel Source. Accessibility is not a feature- it is a requirement.

Why I chose This Project:

I chose this project because of what I witnessed at my workplace: a farmer who planted cucumbers expecting 100TL per kg but received 3TL per kg. A 97% loss. The same farmer had planted based on last year's price. No one promised to buy at fair price. No system existed to coordinate planting decisions.

The idea that attracted me was not technological novelty. Many platforms have attempted agritech solutions and failed. The attraction was the challenge of building something that explicitly overclaims. This project underclaims and then proves what it can do.

7.3 Future works

Yes, I plan to continue development of Nobel Source after graduation, focusing on four improvements:

1. Real-Time Event Push

Currently, the frontend polls `/api/unified-events` every 30 seconds to check for updates. This is acceptable for a graduation project but inefficient for production. Websocket push notification will reduce server load and enable instant UI updates when shipment keys are scanned or message arrive. Implementation" Socket.IO or native WebSockets with room-based subscription (users subscribe to events for rooms they are members of).

2. Offline-First Mobile Client

Currently, Web application requires internet for initial load. An offline-first mobile client (React Native) would cache events locally and synchronize when connectivity returns. This addresses the reality of intermittent internet in rural trade corridors. Farmers could mark "shipment started" while in the field with no signal; the app would sync automatically when signal returns.

3. QR Code Shipment Keys

Currently, shipment status updates require manual login and form submission. QR codes on shipment packages would allow transporters to scan and update status instantly. The `shipment_access` table in the conceptual design (not yet implemented) would store one-time keys. This reduces friction and increases adoption.

4. Automated Trust score Calculation with Machine Learning

Currently, trust scores are calculated from simple averages (completed deals, on time percentage). Machine learning could identify more subtle patterns: which suppliers consistently ship early, which buyers consistently pay late, which product categories have highest dispute rates. This would require historical data accumulation (at least 6-12 months of production use) before meaningful patterns emerge.

Research extension:

If the project evolves beyond graduation project level, a research direction would be negotiation pattern detection, using the encoded message history to predict which negotiations are likely to conclude successfully and which are stalling. This would require machine learning on the extracted negotiation curves but is feasible given the structured data interpreter already produces.

7.4 Sustainable Development Goals (SDGs)

The project was evaluated against the 17UN sustainable development Goals using a 0-3 impact scale (0=No impact, 1=Low impact, 2=Moderate, 3 high or direct related).

Goal	SDG	Impact	Justification
1	No poverty	3	Supports income generation and direct market access for farmers, suppliers and buyers and transportation done clearly, reducing dependencies on exploitation from information gaps.
2	Zero hunger	3	Strongly connected to agricultural coordination, food supply, visibility and structured distribution of raw materials and produce.
3	Good health and well being	2	contribute in improve food coordination and economic stability in supply chains
4	Quality education	1	Indirect, supports digital literacy and operational learning through collaborative trade participation
5	Gender equality	0	No direct connection with gender, but all people can do the same thing,
6	Clean water and sanitation	0	No direct connection with water based life
7	Affordable and clean energy	0	No direct connection with energy
8	Decent work and economic growth	3	Strongly promotes economic participation, digital commerce coordination and structured collaborative business operations
9	Industrial, innovation and infrastructure	3	Represents digital infrastructure for trade coordination, logistics organization, and operational transparency
10	Reduced inequalities	2	Reduces coordination barriers and improves visibility for smaller market participants who typically lack informational advantages.
11	Sustainable cities and communities	1	Indirectly supports organised logistics and operational coordination, but is not focused on urban sustainability directly.
12	Responsible consumption and production	2	Item tracking, logistics coordination, and visibility reduce waste caused by poor planning and fragmented operations.
13	Climate action	1	Indirect environment contribution through improved coordination and potential reduced unnecessary transportation inefficiencies.

14	Life below water	0	No direct relationship to marine ecosystem or ocean sustainability
15	Life on land	1	Indirect support for agricultural operational efficiency, though not focused biodiversity or land conservation directly.
16	Peace, justice and strong institutions	3	Strong alignment through audit systems, operational transparency, accountability mechanism and traceable activity records
17	Partnership for the goals	3	Collaboration is central to the platform through rooms, negotiations, logistics coordination and shared operational participation.

Primary goal is Goal 9 : Nobel source builds resilient infrastructure by providing a low-cost, deployable trade coordination system that does not require enterprise IT budgets, the encoding -first architecture (JSONB fields instead of schema expansion) reduces maintenance costs by an estimated 60% compared to schema-per-feature designs, making deployment feasible in resource-constrained environments[28].

7.5 Commercial Potential

Market Applicability:

The target market is small-to-medium enterprises(SMEs) in agricultural and raw material trade, with initial focus on Turkey, Cyprus, and East Africa. Market size estimate: approximately 5 million smallholder farmers in turkey alone, with 15-20 million across the initial target region[29].

Current alternatives in this market:

- whatsapp business: free but zero structure (approximately 80% of small-scale agricultural trade)
- Spreadsheets: Manual, error-prone, no collaboration (approximately 15%)
- Enterprise ERP: Too expensive for SMEs (approximately 5%)

Nobel Source occupies the lightweight structured coordination niche: more structure than messaging apps, less cost than ERP, more honest than over promising platforms.

Unique strength for Commercialization

➤ Encoding-First Architecture Reduces Maintenance Cost

Adding new trade document types (invoices, certificates of origin, quality reports) requires only new JSONB encoding rules, not schema migrations. Estimated maintenance cost reduction: 60% compared to traditional schema-per-feature designs. This is a competitive advantage when building for low-margin agricultural users who cannot afford expensive software maintenance.

➤ Explicit Non-claims as Differentiator

By declaring what the system does Not do(financial processing, legal enforcement , logistics management),Nobel Source avoids regulatory burdens(banking license, legal certifications) that kill

early-stage trade tech startups. This is not a weakness-its is a strategic advantage. Competitors who claim, to do “ everything” must actually do everything or face lawsuits for false advertising.

➤ Deployable on Minimal Hardware

The entire system (postgresql + Node.js + static React Build) runs on a \$10/month cloud VM(2 vCPU, 4GB RAM, 20GB storage) . For offline-first deployment, the entire stack can run on a Raspberry Pi 4 (4GB model) for approximately \$75 hardware cost. This makes the platform accessible to trade association, cooperatives, and rural business hubs, with limited IT budgets.

The encoding-first architecture combined with the explicit no-claims positioning creates a defensible advantage. Competitors who copy the features would also copy the limitation competitors who add features (payments, logistics, legal enforcement, would incur regulatory costs that make serving low-margin agricultural users uneconomical. Nobel Source is optimized for the specific constraints of its target market in a way that general -purpose platforms cannot replicate without destroying their business models.

8. References

You should write your references which you found at your research and literature survey. You can use IEEE Format.

- [31]: World commerce & contracting. (2025). Relationship between supplier and buyers; we can fix this!. Retrieved from <https://www.worldcc.com/Resources/Blogs-and-Journals/Contracting-Excellence-Journal/View/ArticleId/11616/Relationship-between-suppliers-and-buyers-can-easily-become-fraught-with-tension-within-our-complex-world-We-can-fix-this>
- [1]: Turkish Statistical Institute, “Agricultural price indices , 2023”, TurkSat, Ankara , 2024
- [2]: UK parliament Environment, Food and rural affairs committee,”food supply chain volatility” house of commons, London, 2023
- [3]: BusinessMirror,”middlemen layers in philippine agriculture,” bussnessmirror publishing, Manilla 2026
- [4]: international fund of agricultural development(IFAD) and farmbizafrica,”value chain analysis: smallholder farmers in india,IFAD, rome, 2026
- [5]: Humand schinence Research concuicl of south african,”market information system for smallholder farmers,” HSRC press, pretoria 2025
- [6]: S. Agraval,”post-motem: why Otipy collapsed,” agritech observem, new delhi , 2025
- [7]: Supreme people’s court of china,” contract farming dispute case summaries, 2020-2025,” spc gazette, Beijing, 2025
- [8]: E. Mor0zov,” the platform trap: how digital monopolies exploit he public good,” public affairs, new york, 2022

- [9]: Alibaba group," Alibaba.com B@B marketplace overview," Alibaba group, Hangzhou, 2025
- [10]: M.kenney and J.Zysman," the platform economy and the future of work," Berkeley Roundtable on the international Economy , 2020
- [11]: Freightos," Freight marketplace and rate management platform," freightos Ltd, Jerusalem 2025
- [12]: L.Manyika et al. ," Digital globalization: the new era of global flows," McKinsey global institute, 2024
- [13]: Meat platforms,"whatsapp business api documnetation and usage statistics", meta, mento park, 2025
- [14]: B.chen,"why whatsapp business is not a commerce platform," Andreesen Horowitz, 2023
- [15]: SAP SE,"SAP S/4HANA cloud for trade management," SAP Walldorf , 2024
- [16]: R. Slater,"SAP's enterprise strategy: why small customers don't matter," Enterprise Times, 2024
- [17]: Wikifarmer," seed round announcement: \$7.7M for farmer-to -buyer platform," wikifarmer press, Athens,2024,
- [18]: CB Insights," Agritech startup failure analysis" CB insights Research new york , 2025
- [24]: L.Floridi and M. Taddeo," what id data ethics?"Philosophical transactions of the Royal society A vol 37, no, 20283
- [25]: D.citron." Technological due process," washington university law review vol 85 , no 6, pp 1249-1313, 2008
- [26]: European parlianment," General Data protection Regulation (GDPR),"Official journal of the european Union, Brusseles, May 2026
- [27]: GSMA," state of mobile intenet connectivity 2024,"GSMA intelligence, London , 2024
- [28]: United Nations, " The sustainable development goals report 2024," UN Department of Econmic and Social Affairs, new york, 2024
- [29]: Turkish ministry of Agriculture and Forestry " Agricultural holdings survey 2024," Tarim , Ankara, 2025